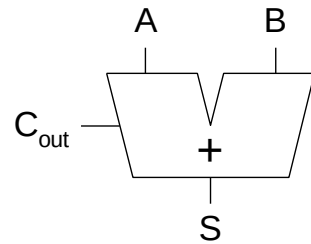


Circuiti aritmetici e memorie

1-Bit Adders

**Half
Adder**

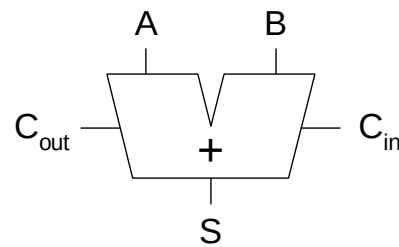


A	B	C _{out}	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$$S = A \oplus B$$

$$C_{\text{out}} = AB$$

**Full
Adder**



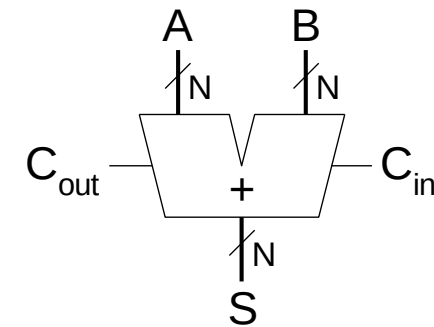
C _{in}	A	B	C _{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$S = A \oplus B \oplus C_{\text{in}}$$

$$C_{\text{out}} = AB + AC_{\text{in}} + BC_{\text{in}}$$

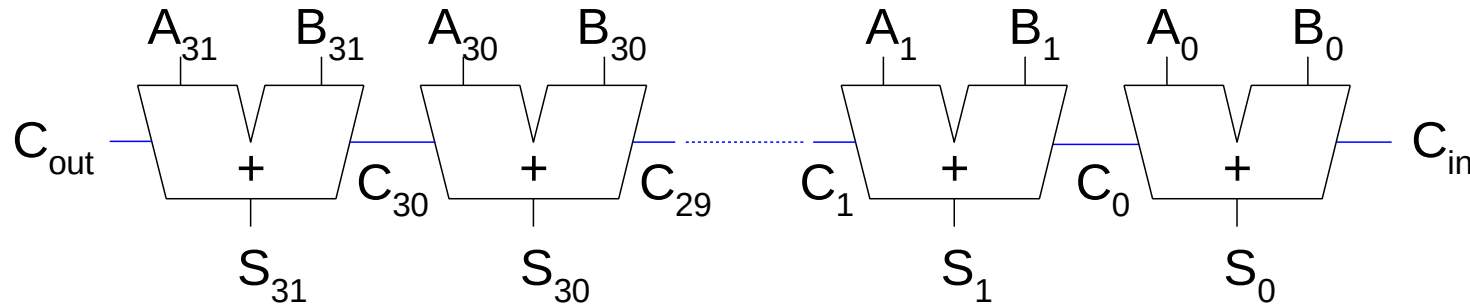
Multibit Adders (CPAs)

- Tipologie carry propagate adders (CPAs):
 - Ripple-carry (lento)
 - Carry-lookahead (veloce)
- Carry-lookahead adders offrono prestazioni migliori ma richiedono più hardware



Ripple-Carry Adder

- 1-bit adder concatenate insieme
- Il riporto si propaga lungo la catena
- Svantaggio: **lento**



Ritardo del Ripple-Carry

$$t_{\text{ripple}} = N t_{FA}$$

t_{FA} è il ritardo di un singolo 1-bit full adder

N è il numero di 1-bit full adders

Carry-Lookahead Adder

Calcolare C_{out} di un blocco di k -bit usando le funzioni *generate* e *propagate*

C_{in}	A_i	B_i	C_{out}
0	0	0	0
1			0
0	0	1	0
1			1
0	1	0	0
1			1
0	1	1	1
1			1

Carry-Lookahead Adder

C_{in}	A_i	B_i	C_{out}
0	0	0	0
1			0
0	0	1	0
1			1
0	1	0	0
1			1
0	1	1	1
1			1

Carry-Lookahead Adder

- Il bit *i-esimo* produce un riporto (carry out) per generazione o per propagazione di un riporto del bit *(i-1)-esimo* (carry in)
- **Generate:** Il bit *i-esimo* genera un carry out se A_i e B_i sono entrambi 1.

$$G_i = A_i B_i$$

- **Propagate:** Il bit *i-esimo* propaga un carry al carry out se A_i o B_i sono 1.

$$P_i = A_i + B_i$$

- **Carry out:** il carry *i* (C_i) è data da:

$$C_i = A_i B_i + (A_i + B_i) C_{i-1} = G_i + P_i C_{i-1}$$

Block Propagate and Generate

Ora dobbiamo estendere le funzioni Propagate and Generate a blocchi di k -bits, ovvero:

- calcolare se un blocco di k -bit $(i, i+1, \dots, i+k-1)$ propaga il carry out del blocco precedente (ovvero da C_{i-1} a C_{i+k-1})
- calcolare se un blocco di k -bit $(i, i+1, \dots, i+k)$ genera un carry out (in C_{i+k-1})

Esempi

0	0	1	0	1	1	1	1	1	1	0	0	0	0	0	0	0
	0	0	1	0	1	0	0	1	0	1	0	1	0	0	0	1
	0	0	1	0	0	1	1	0	1	1	0	0	0	0	0	0
	0	1	0	1	0	0	0	0	0	0	0	1	0	0	0	1

0	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0
	0	0	1	0	1	0	0	1	0	1	0	1	0	0	0	1
	0	0	1	0	0	1	1	0	0	1	0	0	0	0	0	0
	0	1	0	0	1	1	1	1	1	0	0	1	0	0	0	1

Esempi

0	0	1	0	0	0	1	1	1	1	0	0	0	0	0	0	0
	0	0	1	0	1	0	0	1	0	1	0	1	0	0	0	1
	0	0	1	0	0	0	1	0	1	1	0	0	0	0	0	0
	0	1	0	0	1	1	0	0	0	0	0	1	0	0	0	1

0	0	1	0	1	1	1	0	0	1	0	0	0	0	0	0	0
	0	0	1	0	1	0	1	1	0	1	0	1	0	0	0	1
	0	0	1	0	0	1	1	0	0	1	0	0	0	0	0	0
	0	1	0	1	0	0	0	1	1	0	0	1	0	0	0	1

.

Esempi

0	0	1	0	0	0	1	1	1	1	0	0	0	0	0	0	0
	0	0	1	0	1	0	0	1	0	1	0	1	0	0	0	1
	0	0	1	0	0	0	1	0	1	1	0	0	0	0	0	0
	0	1	0	0	1	1	0	0	0	0	0	1	0	0	0	1

0	0	1	0	0	1	1	0	0	1	0	0	0	0	0	0	0
	0	0	1	0	0	0	1	1	0	1	0	1	0	0	0	1
	0	0	1	0	0	1	1	0	0	1	0	0	0	0	0	0
	0	1	0	0	1	0	0	1	1	0	0	1	0	0	0	1

Block Propagate and Generate Signals

- Block propagate and generate per un blocco di 4-bit ($P_{3:0}$ and $G_{3:0}$):

$$P_{3:0} = P_3 P_2 P_1 P_0$$

$$G_{3:0} = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 = \\ G_3 + P_3 (G_2 + P_2 (G_1 + P_1 G_0))$$

- In generale,

$$P_{i:j} = P_i P_{i-1} P_{i-2} \dots P_j$$

$$G_{i:j} = G_i + P_i (G_{i-1} + P_{i-1} (G_{i-2} + P_{i-2} (\dots G_j) \dots))$$

$$C_i = G_{i:j} + P_{i:j} C_{j-1}$$

Esempio

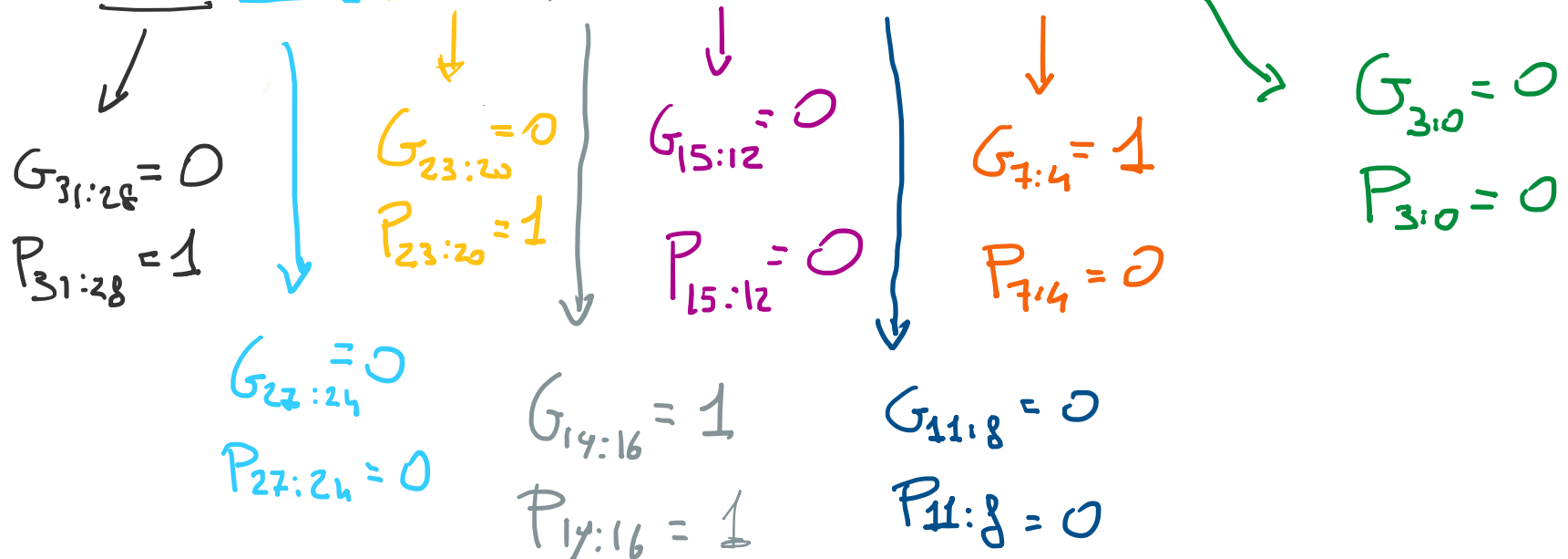
- Consideriamo le due parole a 32 bit

- 0x52A703C1

- 0xA05C11E3

- 0x52A703C1 = 0101 0010 1010 0111 0000 0011 1100 0001

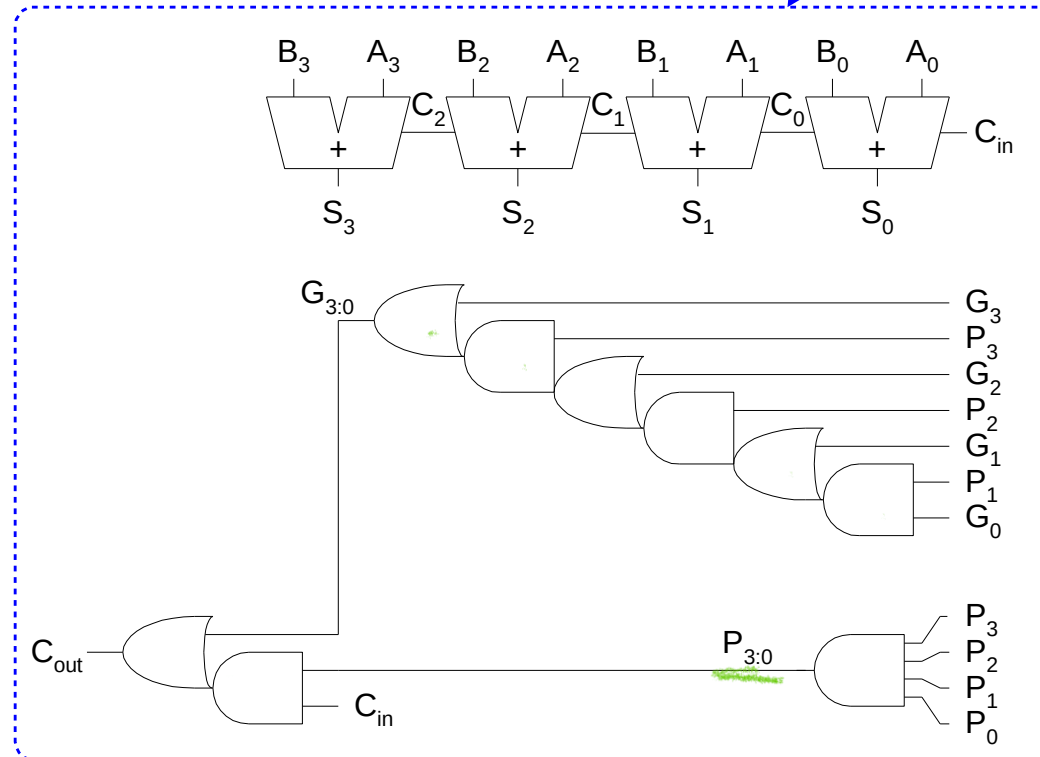
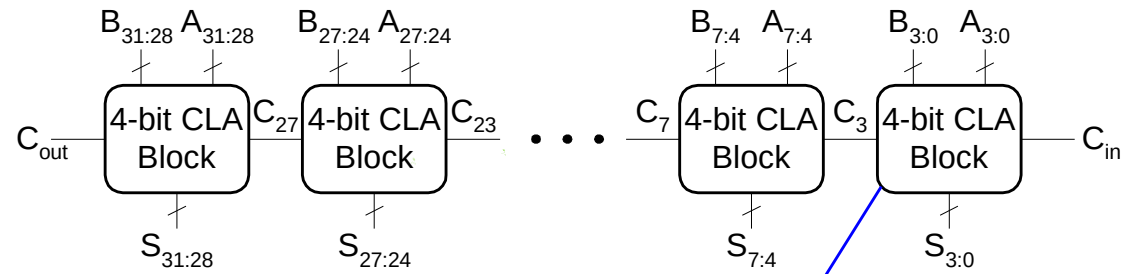
- 0xA05C11E3 = 1010 0000 0101 1100 0001 0001 1110 0011



Carry-Lookahead Addition

- **Step 1:** calcola tutti i G_i and P_i
- **Step 2:** calcola G and P per blocchi di k -bit
- **Step 3:** C_{in} si propaga mediante la logica propagate/generate dei vari blocchi di k -bit

32-bit CLA with 4-bit Blocks



Ritardo del Carry-Lookahead Adder

Per un N -bit CLA con blocchi di k bit:

$$t_{CLA} = t_{pg} + t_{pg_block} + (N/k - 1)t_{AND_OR} + kt_{FA}$$

- t_{pg} : ritardo per generare P_i, G_i
- t_{pg_block} : ritardo per generare $P_{i:j}, G_{i:j}$
- t_{AND_OR} : ritardo delle porte AND/OR a monte della logica propagate/generate, questo ritardo si propaga da C_{in} a C_{out} che si propaga lungo i $N/k - 1$ blocchi

Un N -bit carry-lookahead adder è generalmente più veloce di un ripple-carry adder per $N > 16$

Confronto

- 32-bit ripple-carry vs 32-bit CLA con blocchi di 4 bit
- 2-input gate delay = 100 ps; full adder delay = 300 ps

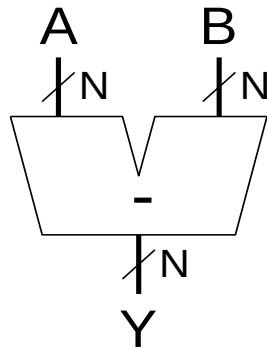
$$\begin{aligned}t_{\text{ripple}} &= Nt_{FA} = 32(300 \text{ ps}) \\ &= \mathbf{9.6 \text{ ns}}\end{aligned}$$

$$\begin{aligned}t_{CLA} &= t_{pg} + t_{pg_block} + (N/k - 1)t_{AND_OR} + kt_{FA} \\ &= [100 + 600 + (7)200 + 4(300)] \text{ ps} \\ &= \mathbf{3.3 \text{ ns}}\end{aligned}$$

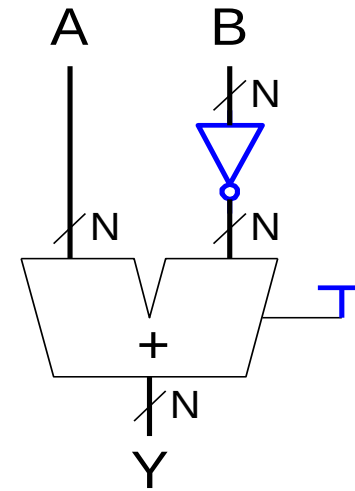
Sottrattore

- Nella rappresentazione a complemento a 2 per complementare un numero occorre invertire ogni cifra e sommare 1
 - Es: $2 = 0010_2 \rightarrow -2 = 1101_2 + 0001_2 = 1110_2$

Symbol

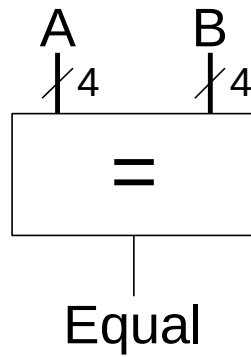


Implementation

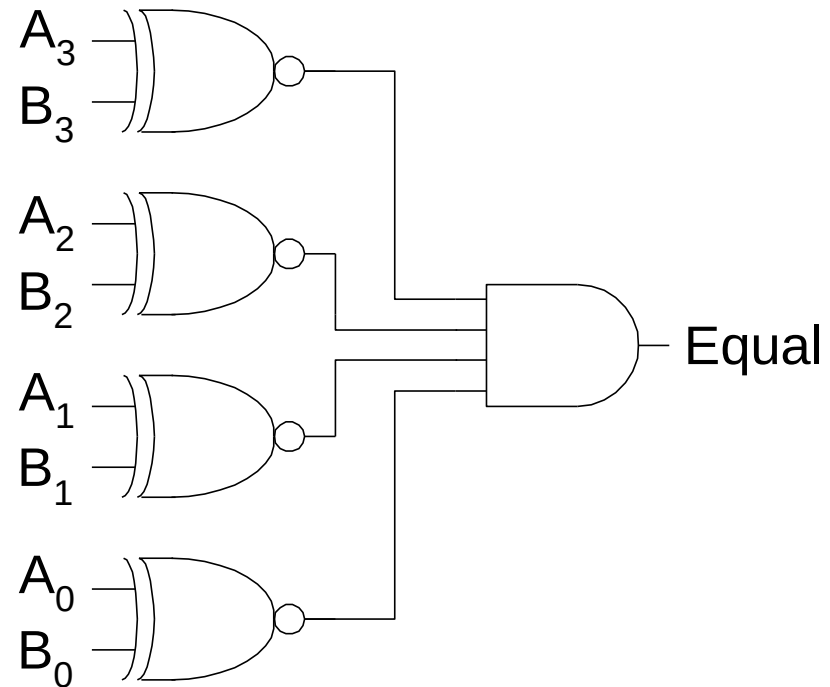


Comparatore: uguaglianza

Symbol

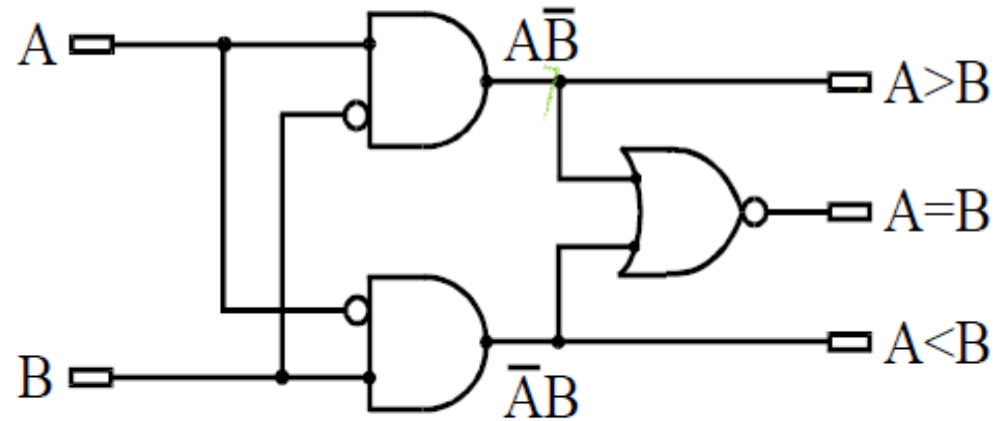


Implementation

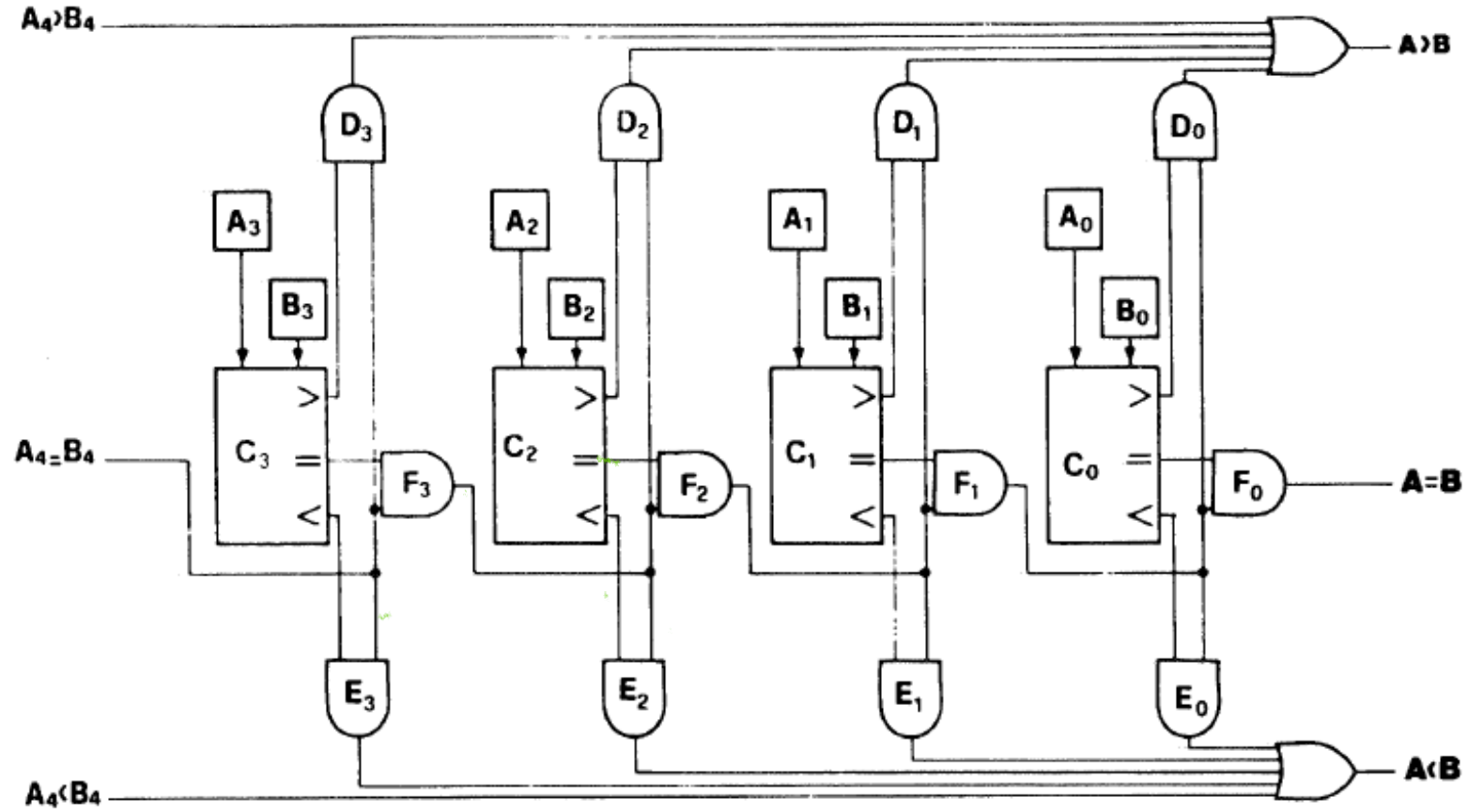


Comparatore completo a un bit

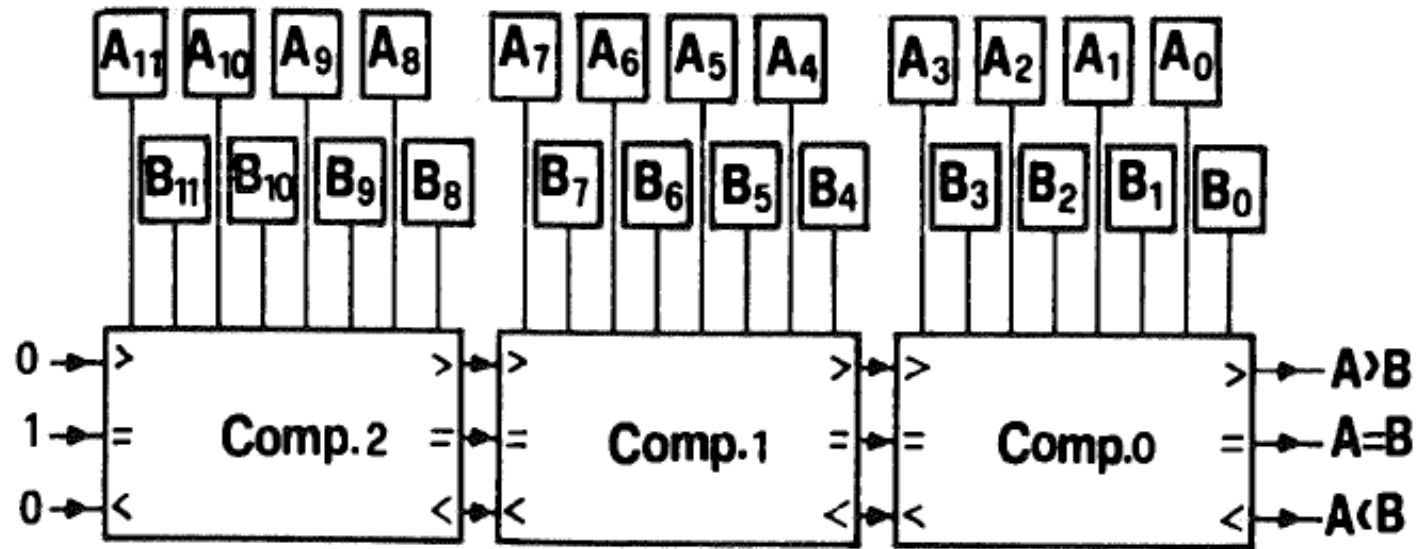
A	B	$A > B$	$A = B$	$A < B$
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0



Comparatore completo a 4 bit



Comparatore completo a 12 bit



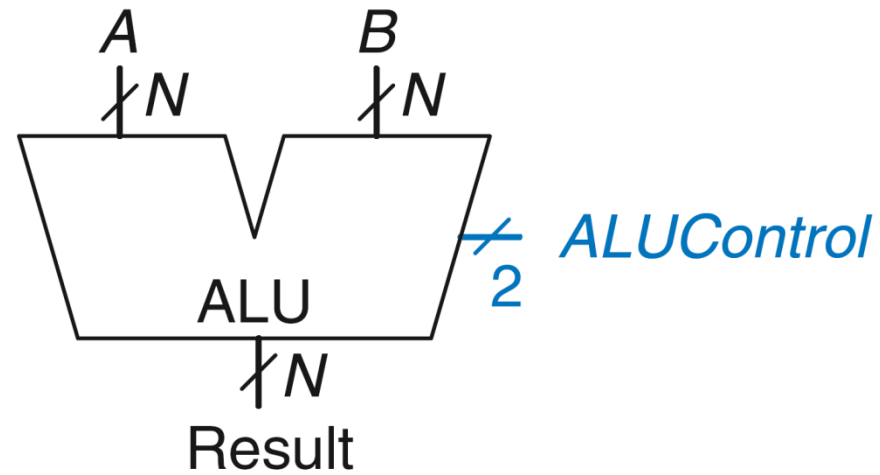
ALU: Arithmetic Logic Unit

Operazioni che ALU tipicamente esegue:

- Addizione
- Sottrazione
- AND (bit a bit)
- OR (bit a bit)

ALU: Arithmetic Logic Unit

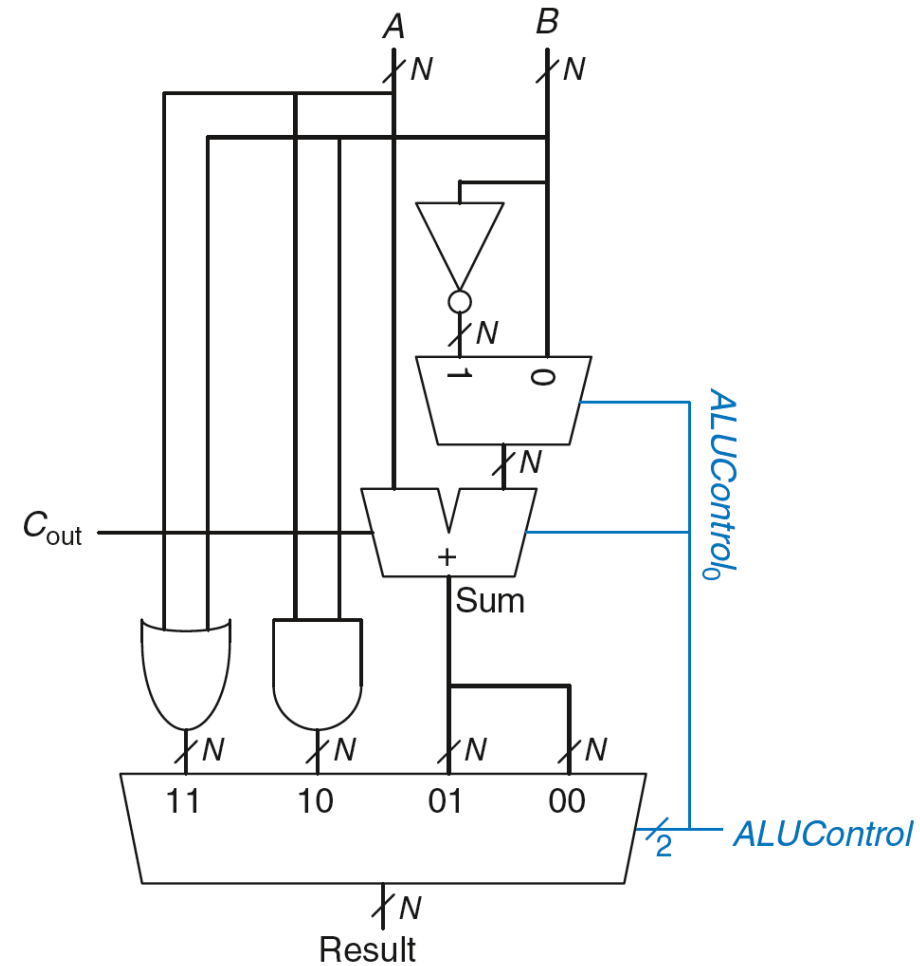
ALUControl _{1:0}	Function
00	Add
01	Subtract
10	AND
11	OR



ALU: Arithmetic Logic Unit

ALUControl _{1:0}	Function
00	Add
01	Subtract
10	AND
11	OR

Esempio: $A \text{ OR } B$

$$ALUControl_{1:0} = 11$$


ALU: Arithmetic Logic Unit

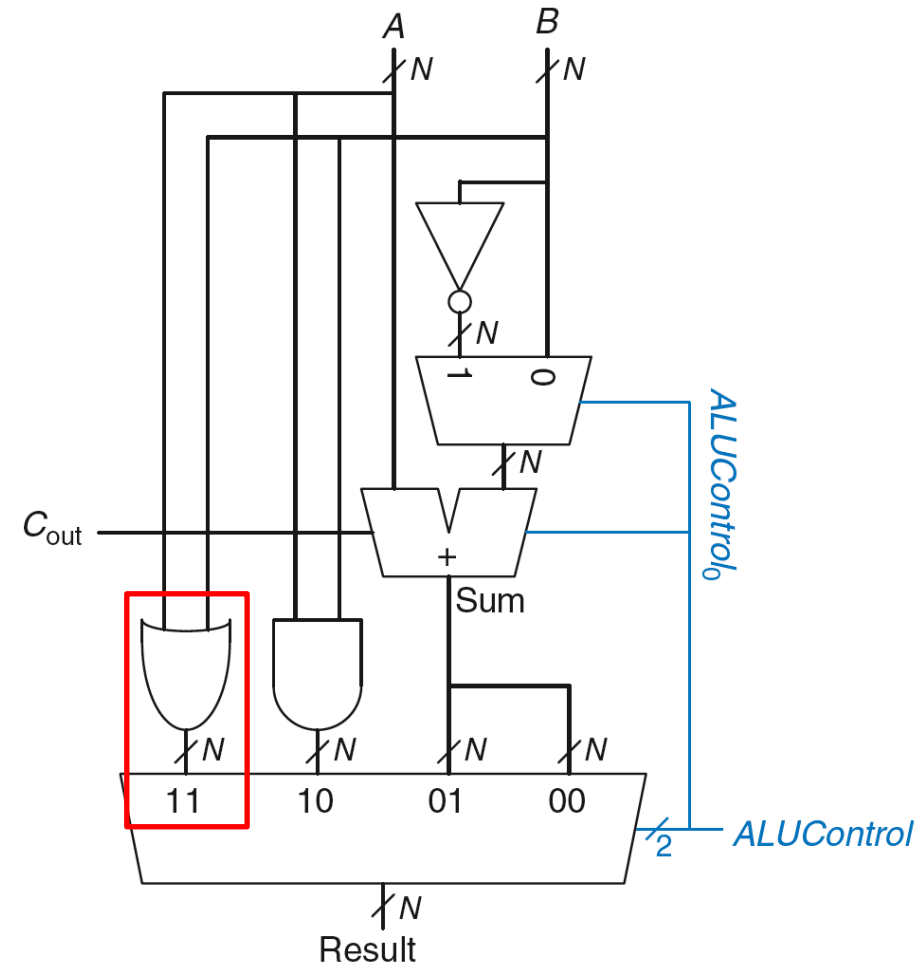
ALUControl _{1:0}	Function
00	Add
01	Subtract
10	AND
11	OR

Esempio: $A \text{ OR } B$

$ALUControl_{1:0} = 11$

Mux seleziona l'output della porta OR

Result = $A \text{ OR } B$



ALU: Arithmetic Logic Unit

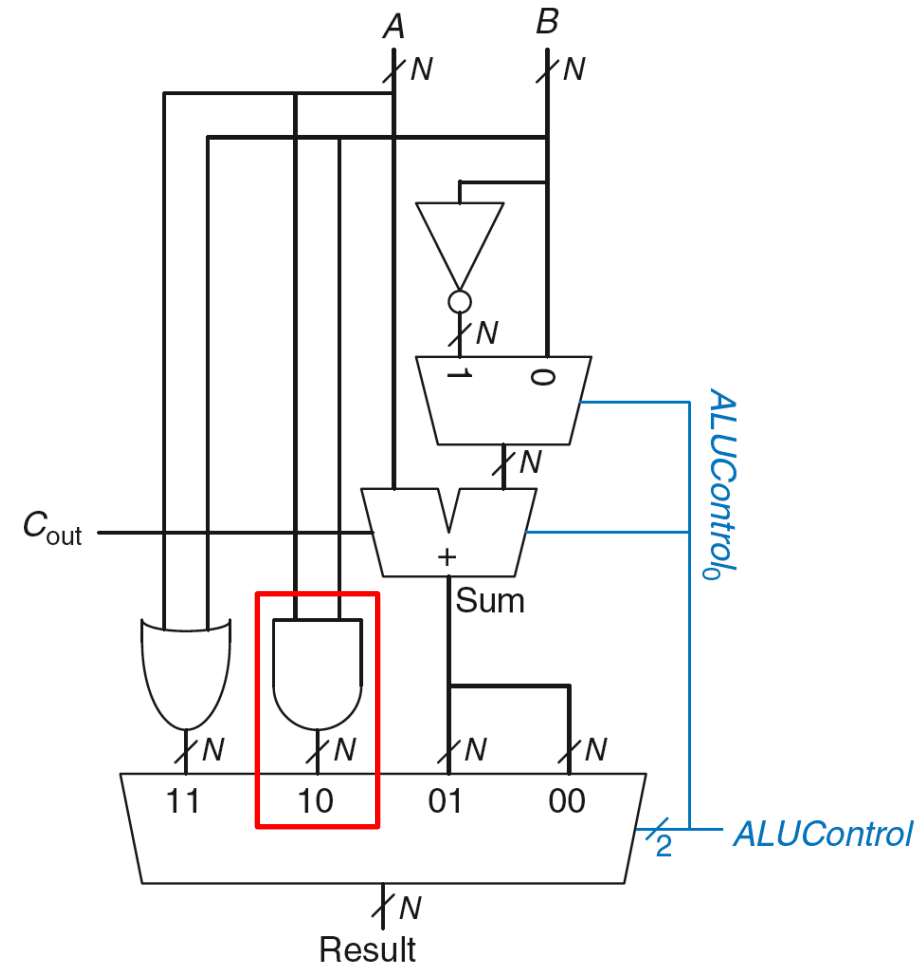
ALUControl _{1:0}	Function
00	Add
01	Subtract
10	AND
11	OR

Esempio: A AND B

$ALUControl_{1:0} = 10$

Mux seleziona l'output della porta AND

Result = A AND B

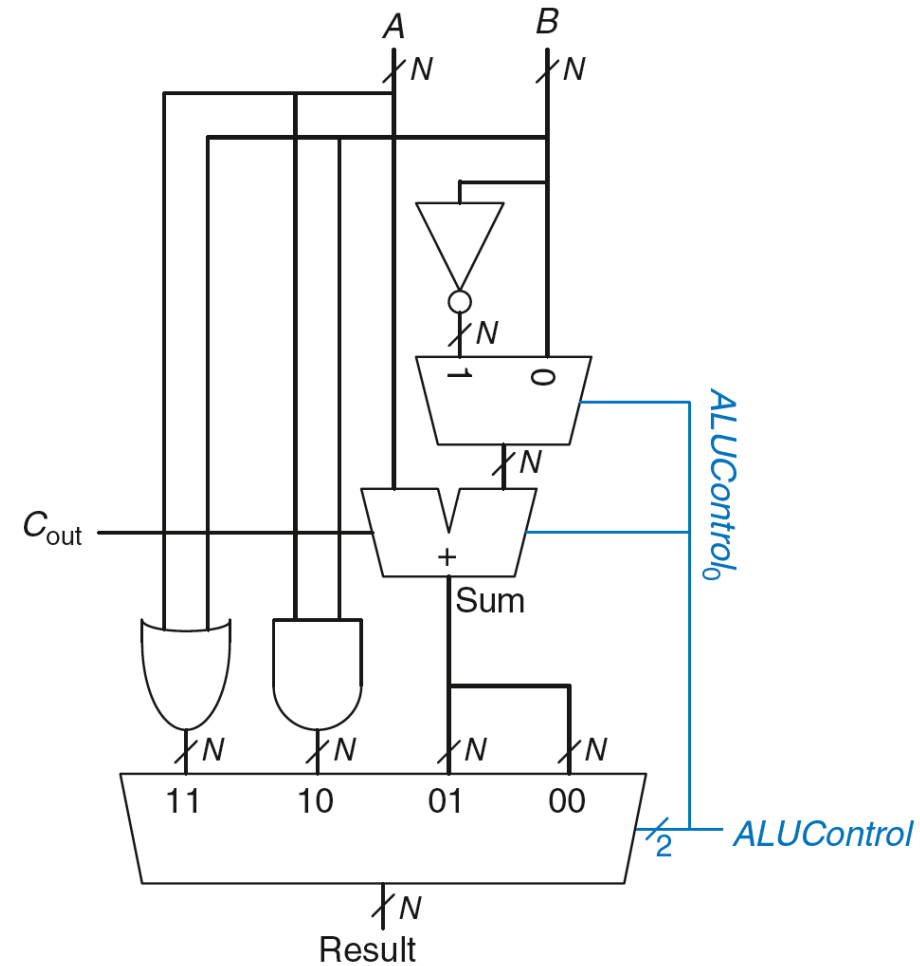


ALU: Arithmetic Logic Unit

ALUControl _{1:0}	Function
00	Add
01	Subtract
10	AND
11	OR

Esempio: $A + B$

$ALUControl_{1:0} = 00$



ALU: Arithmetic Logic Unit

ALUControl _{1:0}	Function
00	Add
01	Subtract
10	AND
11	OR

Esempio: $A + B$

$ALUControl_{1:0} = 00$

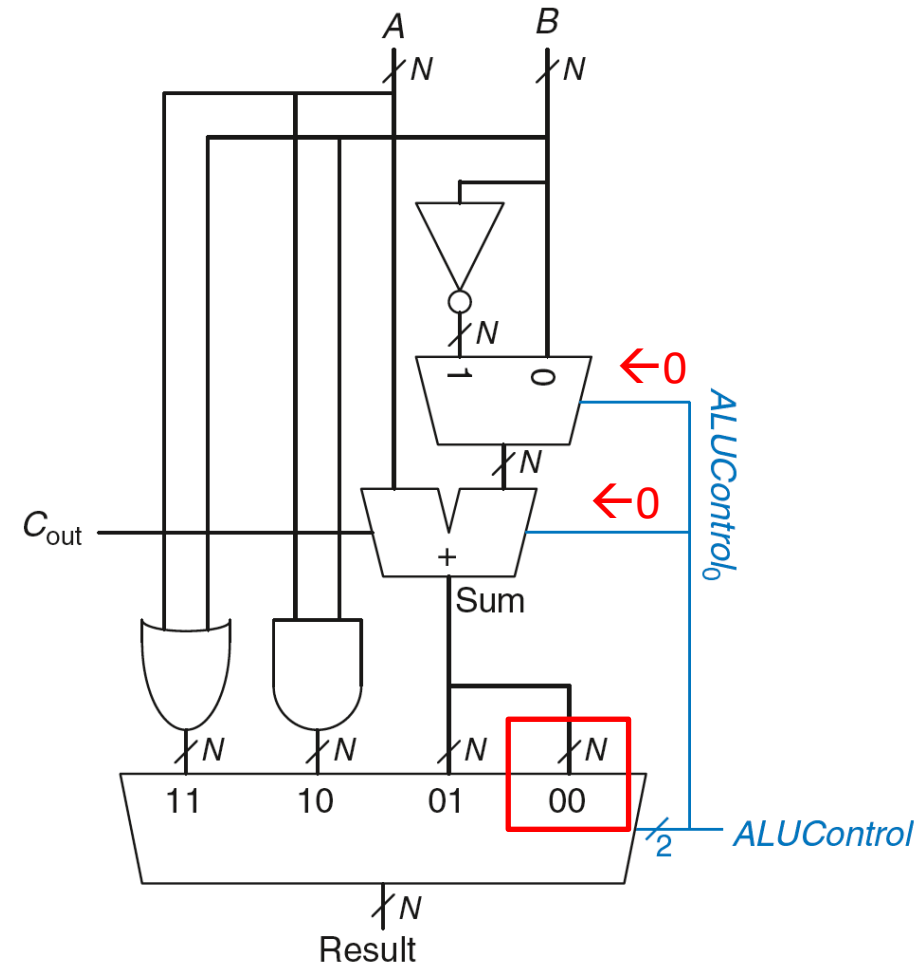
$ALUControl_0 = 0$, quindi:

$C_{in} = 0$

il 2nd input dell'adder è B

Mux seleziona Sum come $Result$

$Result = A + B$



ALU: Arithmetic Logic Unit

ALUControl _{1:0}	Function
00	Add
01	Subtract
10	AND
11	OR

Esempio: $A - B$

$ALUControl_{1:0} = 01$

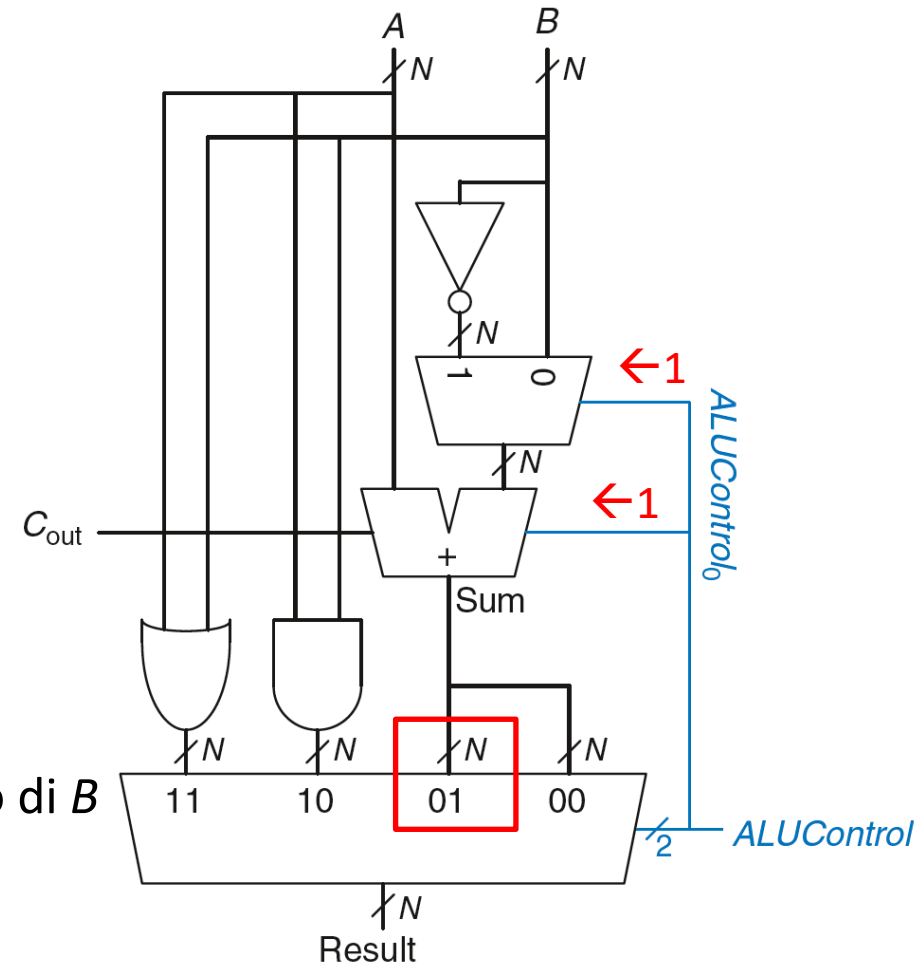
$ALUControl_0 = 1$, quindi:

$C_{in} = 1$

2nd input dell'adder è il complemento di B

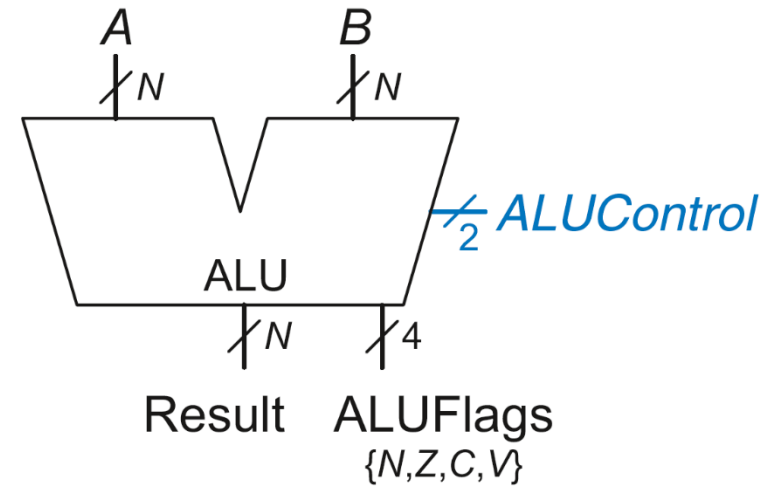
Mux seleziona *Sum* come *Result*

$Result = A - B$

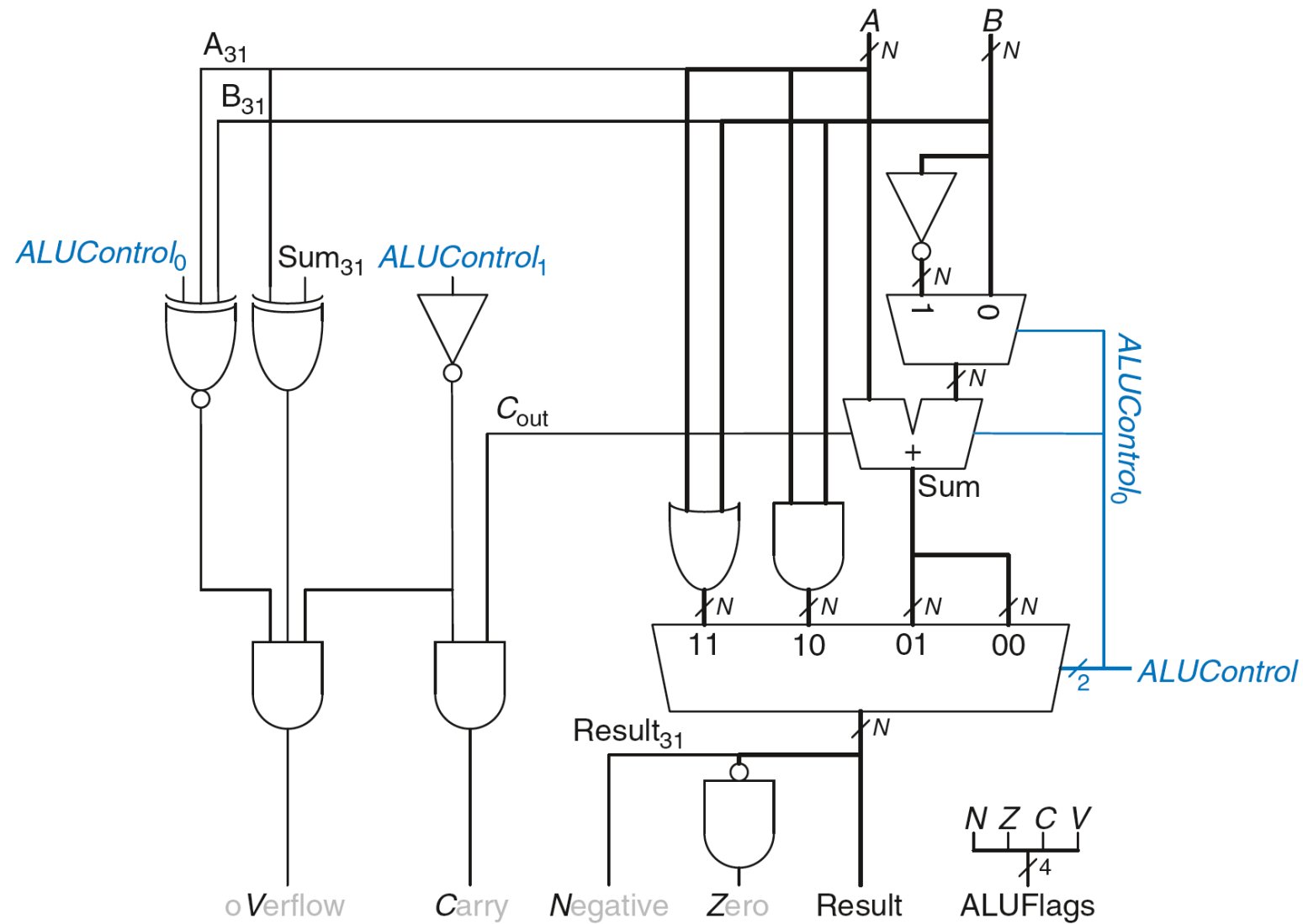


ALU con flags di stato

Flag	Description
<i>N</i>	Result is N egative
<i>Z</i>	Result is Z ero
<i>C</i>	Adder produces C arry out
<i>V</i>	Adder o V erflowed

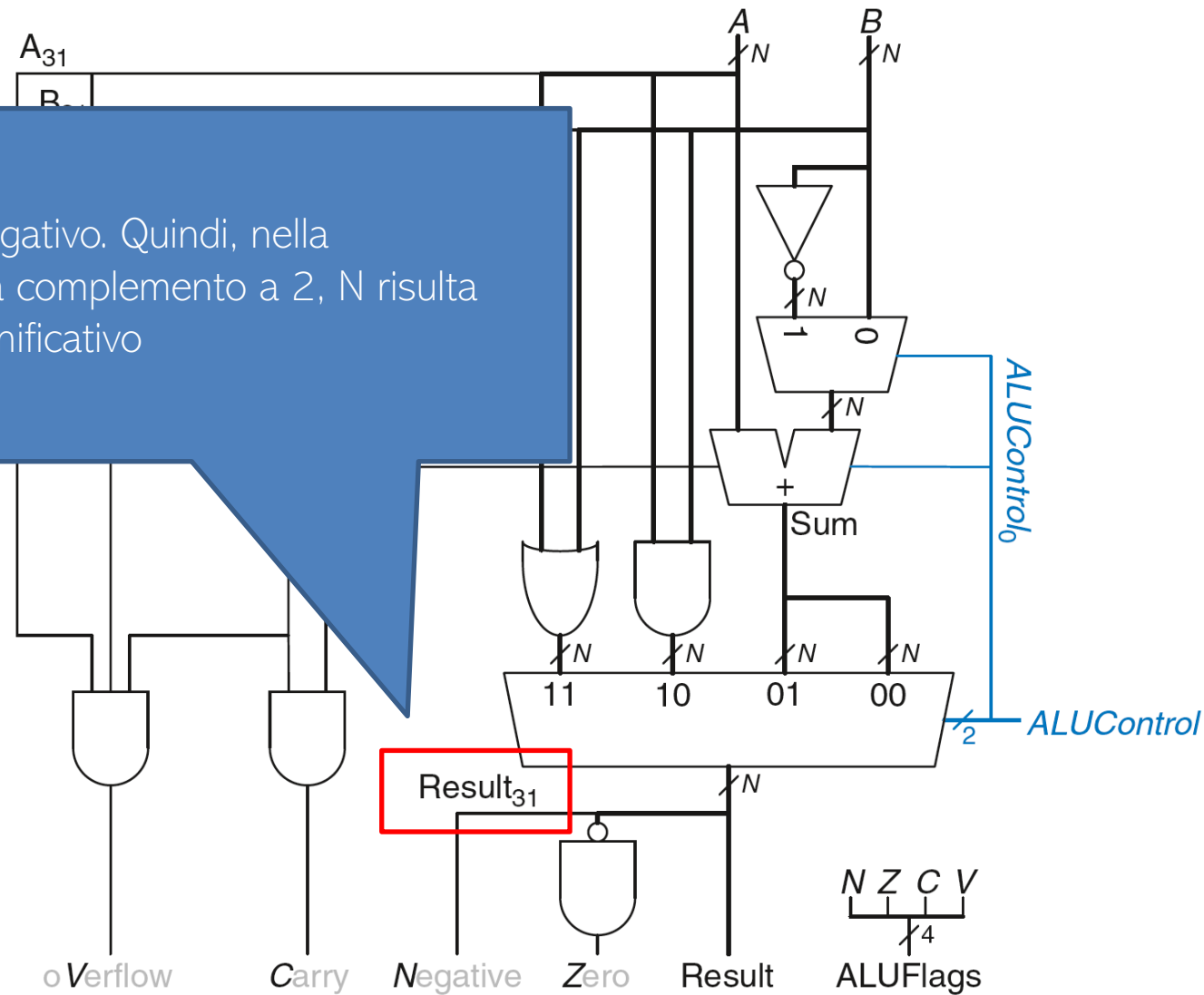


ALU con flags di stato

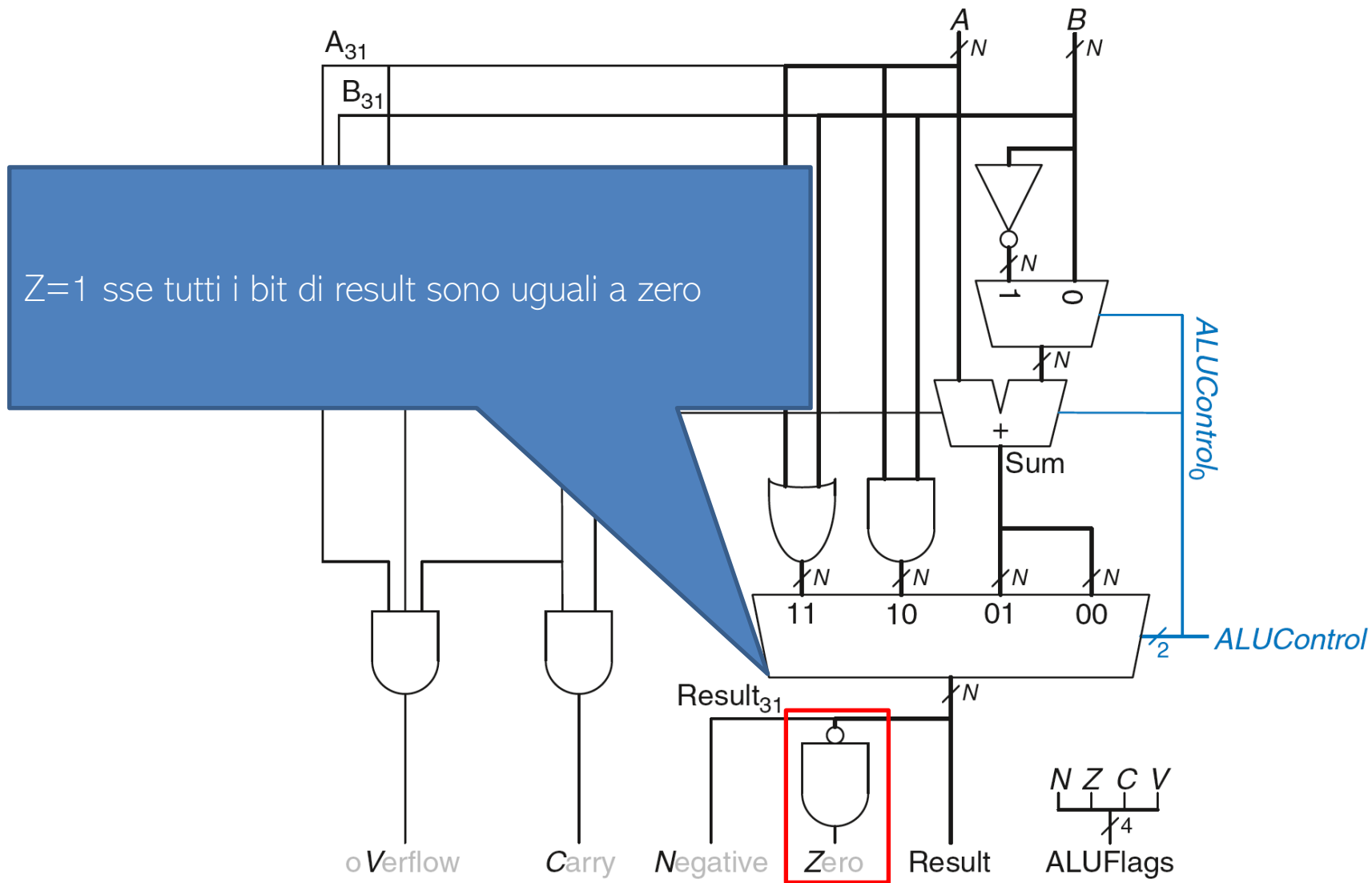


ALU con flags di stato: Negative

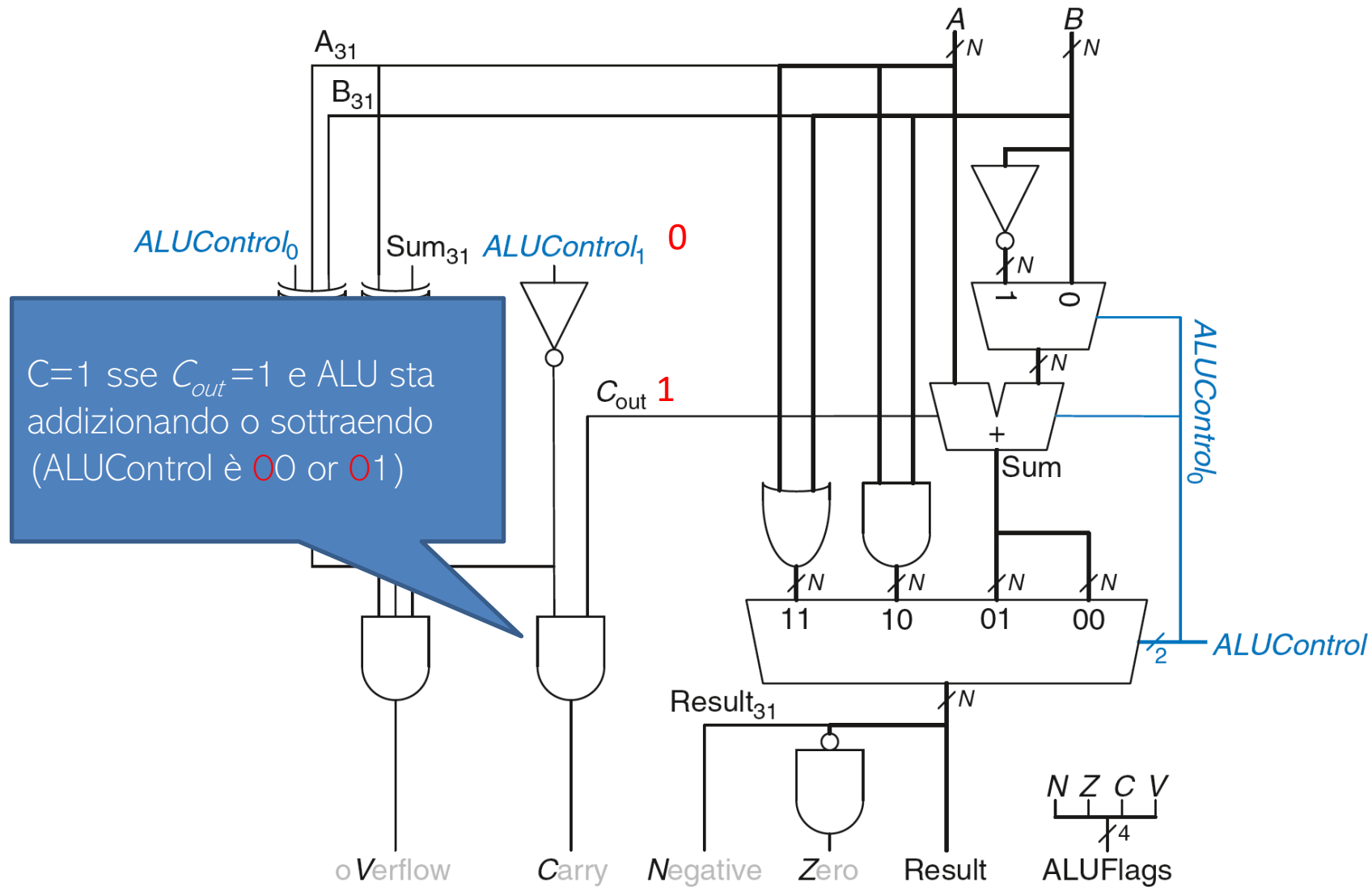
$N=1$ se Result negativo. Quindi, nella rappresentazione a complemento a 2, N risulta essere il bit più significativo



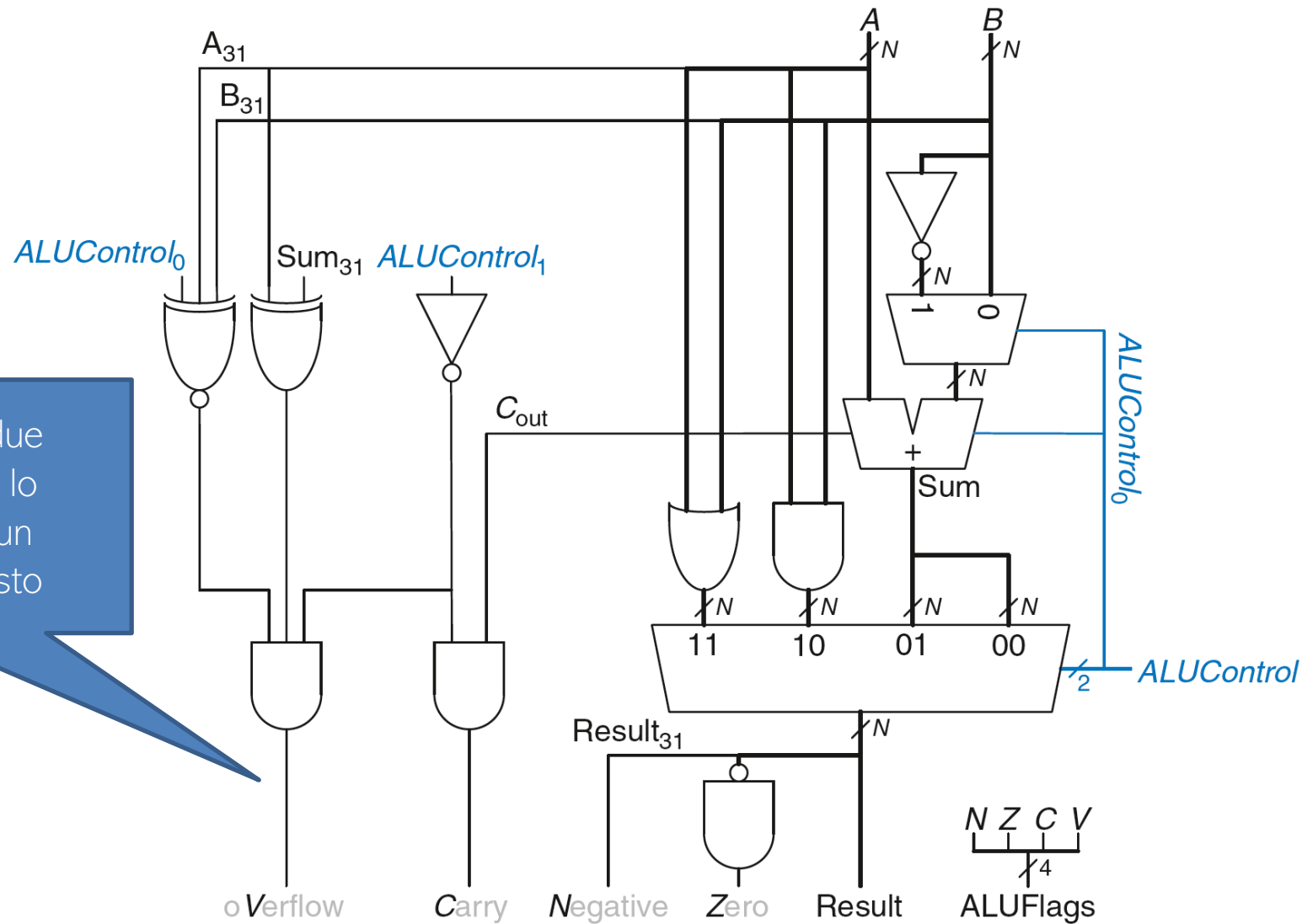
ALU con flags di stato: Zero



ALU con flags di stato: Carry

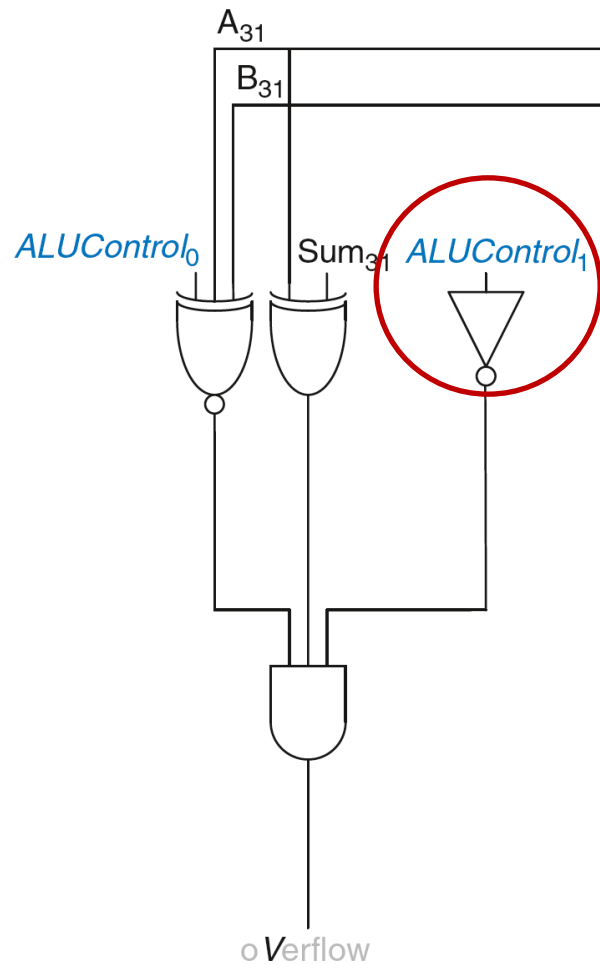


ALU con flags di stato: Overflow



$V=1$ se la somma di due input del somatore con lo stesso segno produce un numero di segno opposto

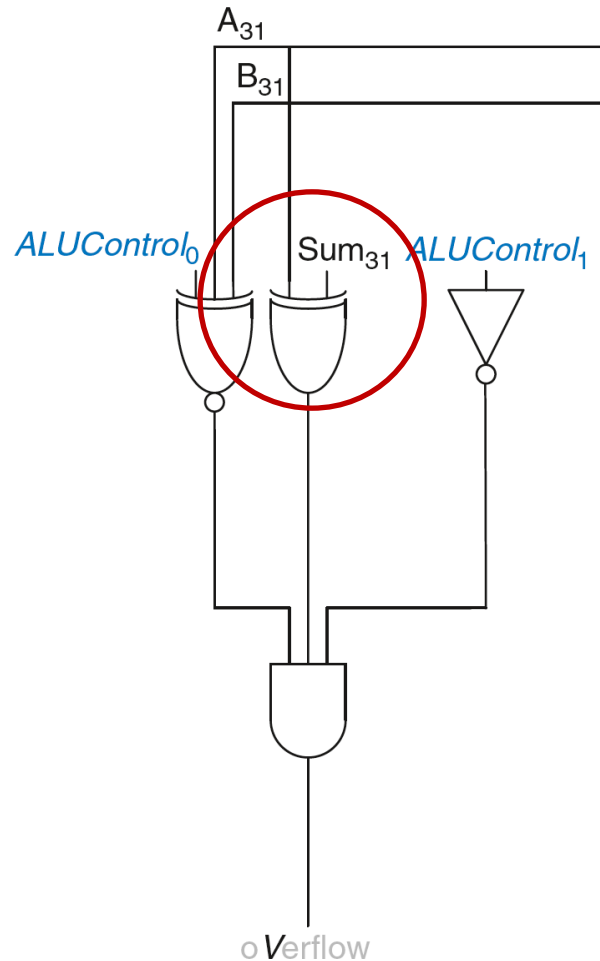
ALU con flags di stato: Overflow



$V = 1$ sse:

ALU esegue una addizione o sottrazione
($ALUControl_1 = 0$)

ALU con flags di stato: Overflow



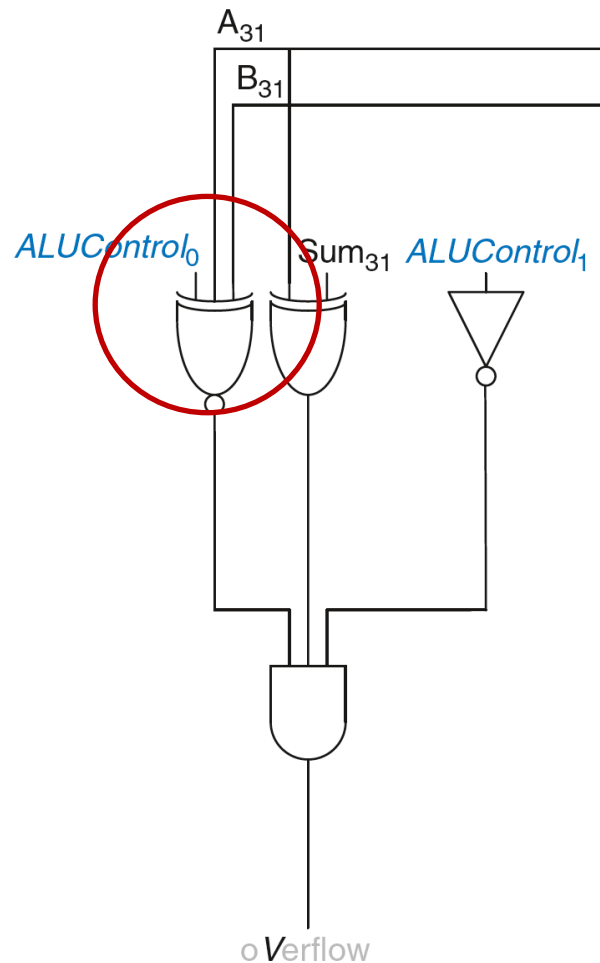
$V = 1$ sse:

ALU esegue una addizione o sottrazione
($ALUControl_1 = 0$)

AND

A e Sum hanno segno opposto

ALU con flags di stato: Overflow



V = 1 sse:

ALU esegue una addizione o sottrazione
($ALUControl_1 = 0$)

AND

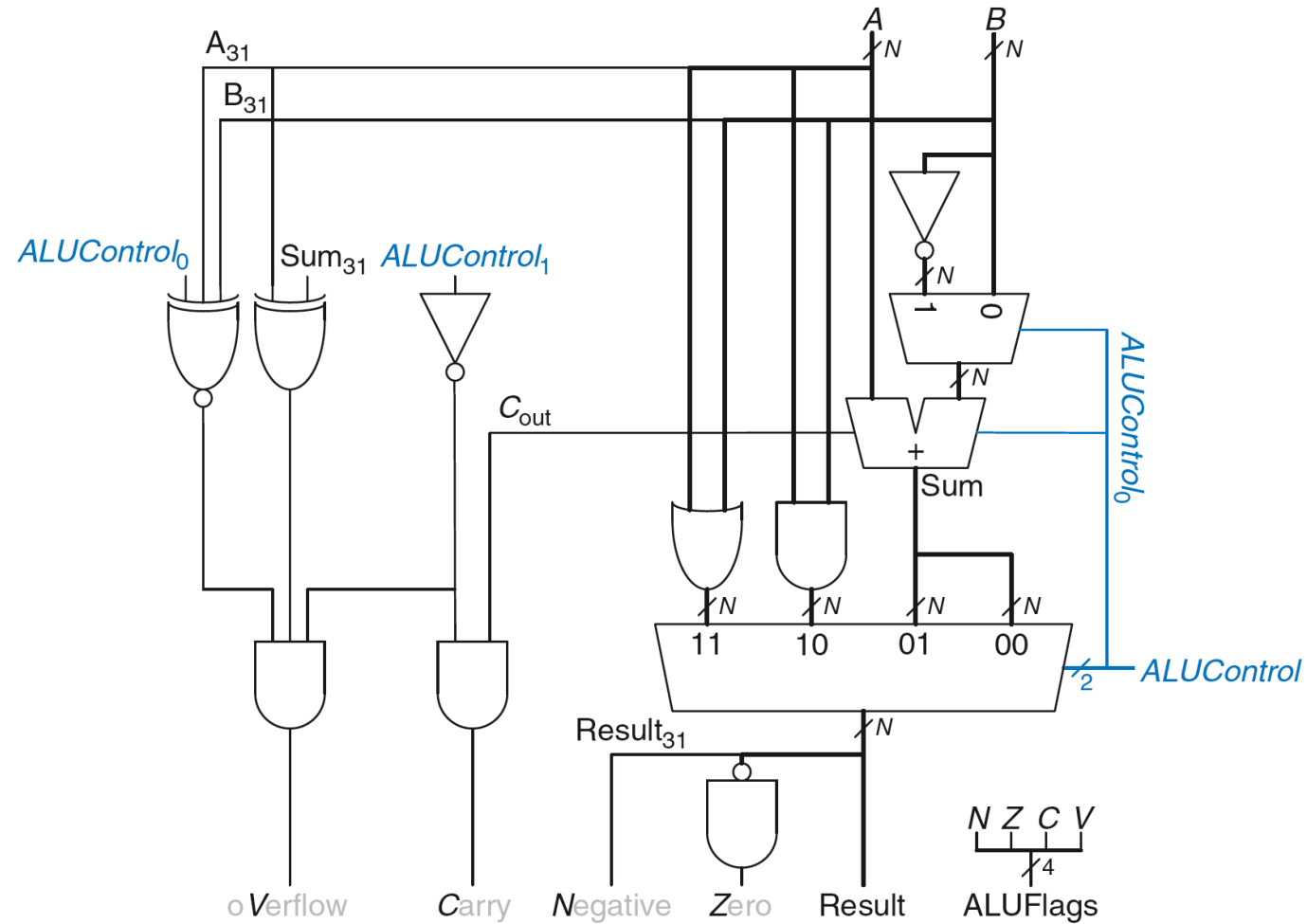
A e Sum hanno segno opposto

AND

A e B hanno lo stesso segno sotto addizione
($ALUControl_0 = 0$) **OR**

A e B hanno segni differenti sotto sottrazione
($ALUControl_0 = 1$)

ALU con flags di stato



Shifters

Logical shifter: trasla i bit a sinistra o a destra e riempie gli spazi vuoti con degli 0

Arithmetic shifter: come il logical shifter a sinistra, nella traslazione a destra invece riempie gli spazi vuoti con il bit più significativo (msb)

Rotator: ruota i bits in cerchio, i bits che “escono” da un lato rientrano dall’ “altro”

Shifters

Logical shifter:

- Ex: 11001 >> 2 = 00110
- Ex: 11001 << 2 = 00100

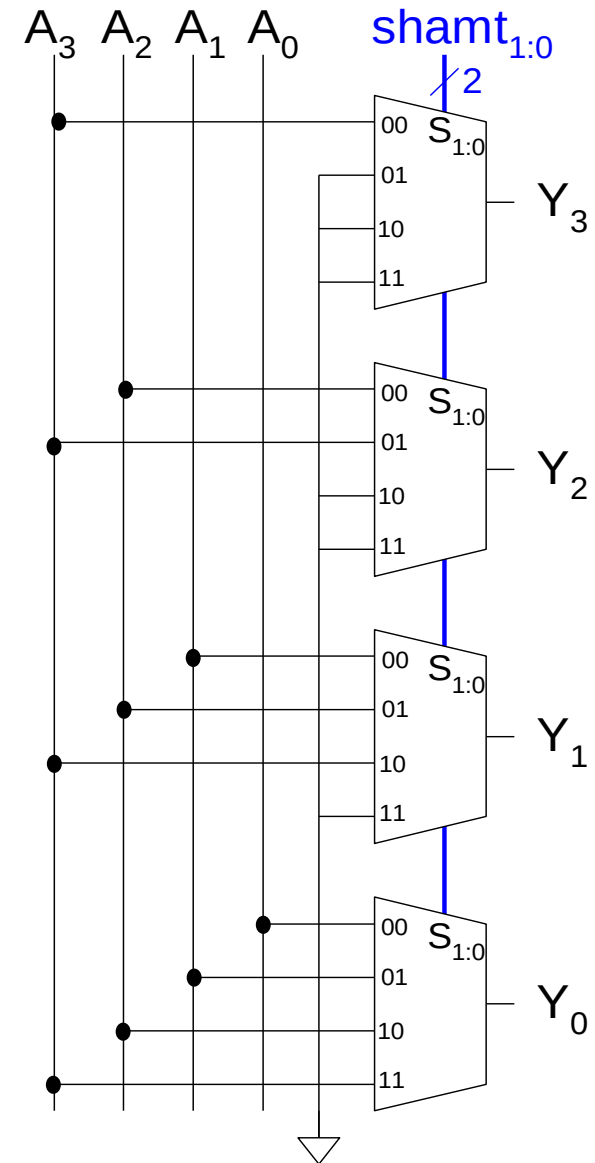
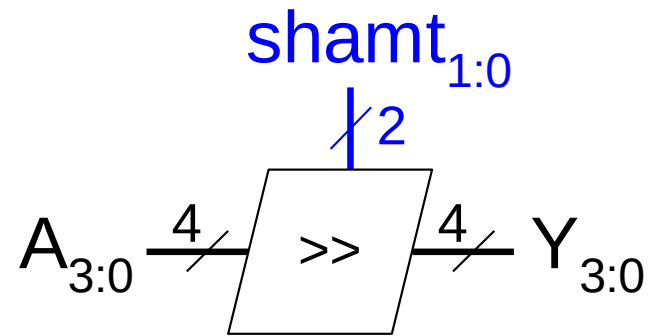
Arithmetic shifter:

- Ex: 11001 >>> 2 = 11110
- Ex: 11001 <<< 2 = 00100

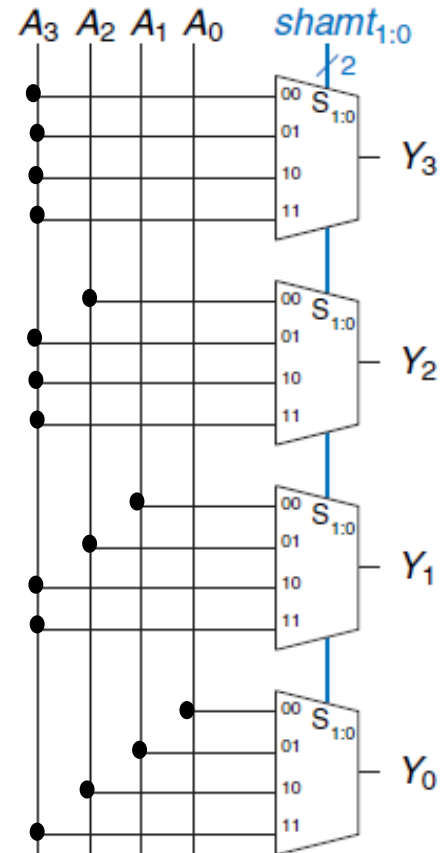
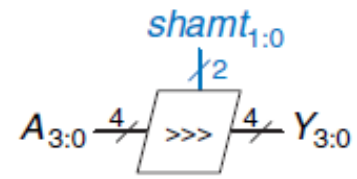
Rotator:

- Ex: 11001 ROR 2 = 01110
- Ex: 11001 ROL 2 = 00111

Shifter Design



Shifter Design



Shifters as Multipliers, Dividers

- $A \lll N = A \times 2^N$

- **Example:** $00001 \ll 2 = 00100$ ($1 \times 2^2 = 4$)

- **Example:** $11101 \ll 2 = 10100$ ($-3 \times 2^2 = -12$)

- $A \ggg N = A \div 2^N$

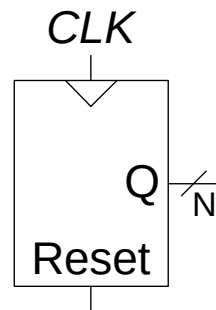
- **Example:** $01000 \ggg 2 = 00010$ ($8 \div 2^2 = 2$)

- **Example:** $10000 \ggg 2 = 11100$ ($-16 \div 2^2 = -4$)

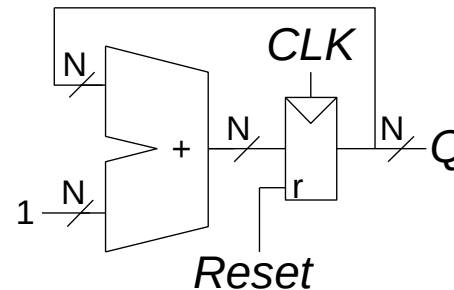
Counters

- Incrementa ad ogni clock
- Usato per eseguire cicli su numeri:
000, 001, 010, 011, 100, 101, 110, 111, 000, 001...
- Si usa:
 - Come orologio digitale
 - Program counter: tiene traccia della istruzione corrente da eseguire

Symbol



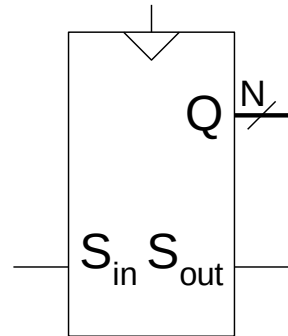
Implementation



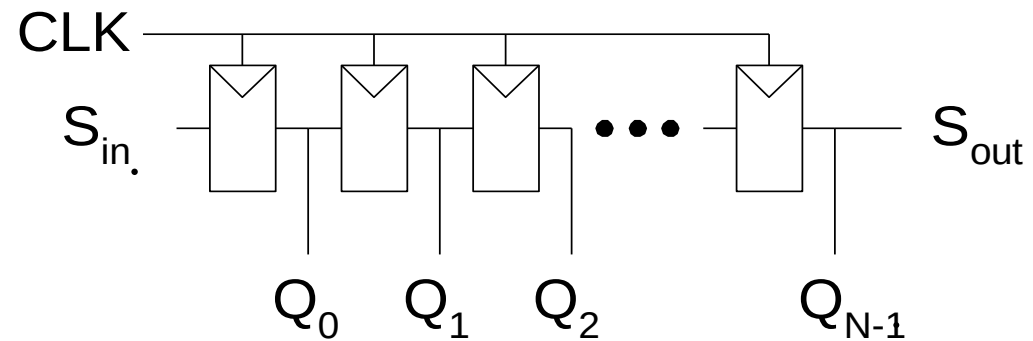
Shift Registers

- Trasla di un bit ad ogni clock
- Ritorna un bit in uscita (S_{out}) ad ogni clock
- *Serial-to-parallel converter*: converte un input seriale (S_{in}) in un output parallelo ($Q_{0:N-1}$)

Symbol:

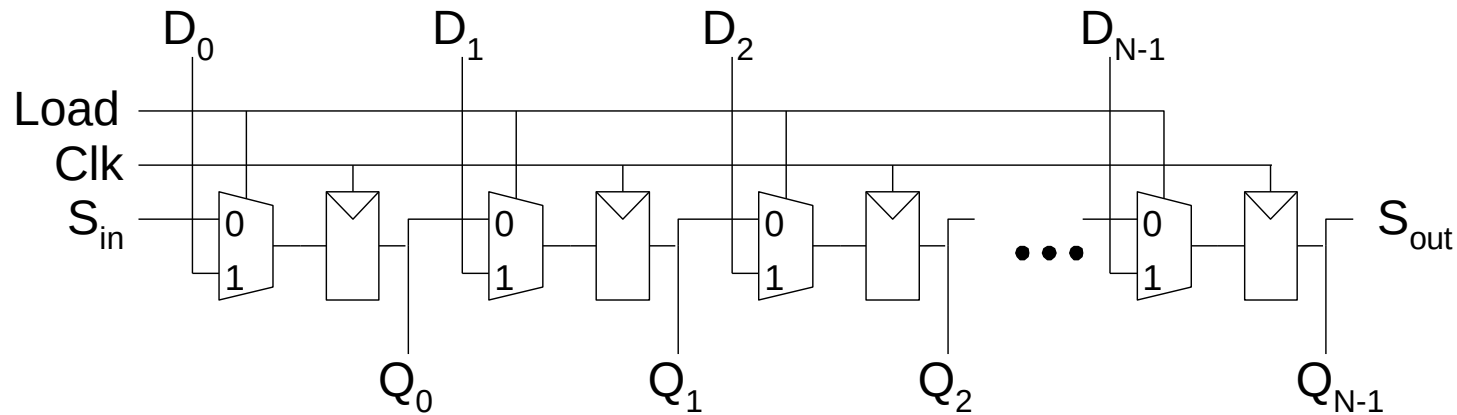


Implementation:



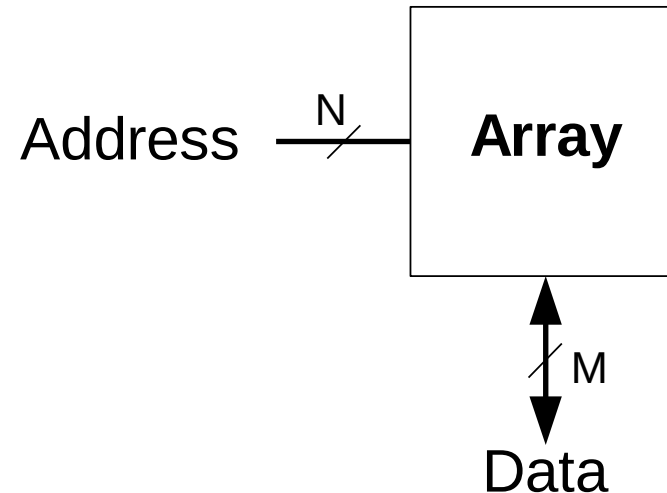
Shift Register con load parallelo

- Quando $Load = 1$, funziona come un usuale registro a N -bit
- Quando $Load = 0$, agisce da shift register
- Quindi può agire da convertitore *seriale-parallelo* (da S_{in} a $Q_{0:N-1}$) o da convertitore *parallelo-seriale* ($D_{0:N-1}$ to S_{out})



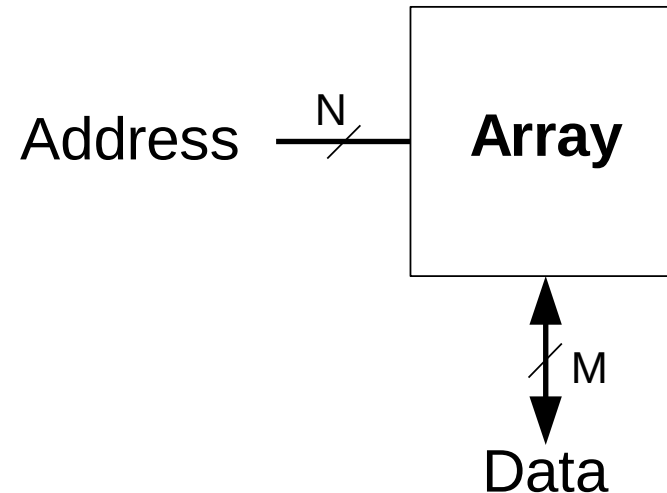
Memory Arrays

- Memorizzare efficacemente una grossa quantità di dati
- 3 tipologie:
 - Dynamic random access memory (DRAM)
 - Static random access memory (SRAM)
 - Read only memory (ROM)
- dato a M -bit letto/scritto su di un unico indirizzo a N -bit



Memory Arrays

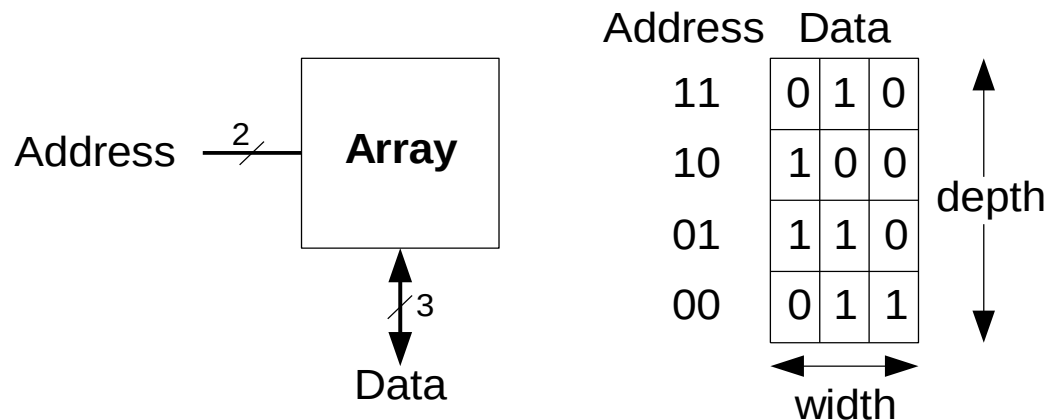
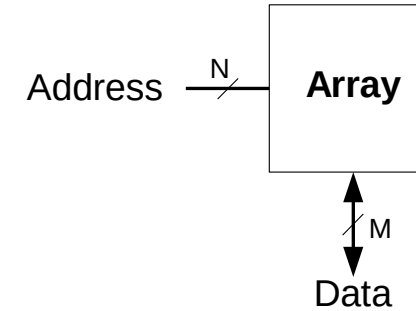
- Memorizzare efficacemente una grossa quantità di dati
- 3 tipologie:
 - Dynamic random access memory (DRAM)
 - Static random access memory (SRAM)
 - Read only memory (ROM)
- dato a M -bit letto/scritto su di un unico indirizzo a N -bit



Tipicamente
 $M=8$ (un byte) e
 $N=32$ (o 64)

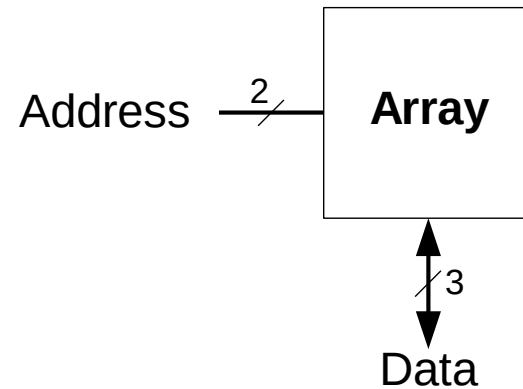
Memory Arrays

- Array bidimensionali di celle di memoria
- Ogni cella memorizza un bit
- Con N bit di indirizzo e M bits data:
 - 2^N righe e a M colonne
 - **Depth:** numero di righe (parole)
 - **Width:** numero di colonne (lunghezza di una parola)
 - **Dimensione Array :** depth $\times$ width = $2^N \times M$



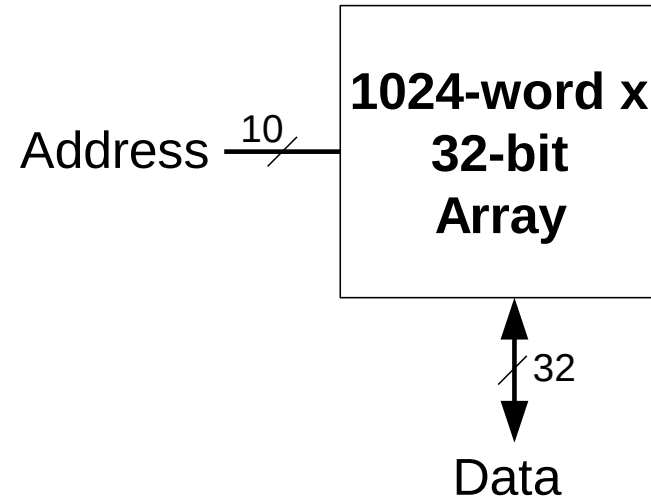
Esempio

- $2^2 \times 3$ -bit array
- Numero parole: 4
- Lunghezza parole: 3-bits
- All'indirizzo 10 corrisponde la parola 100

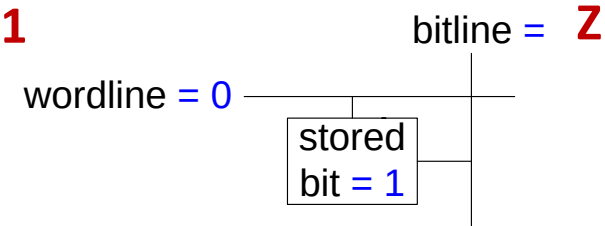
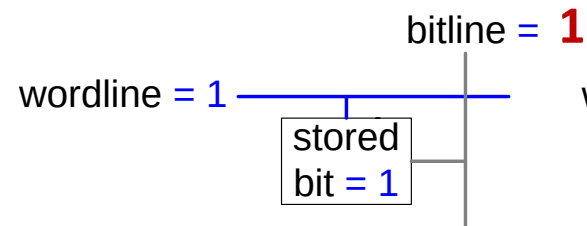
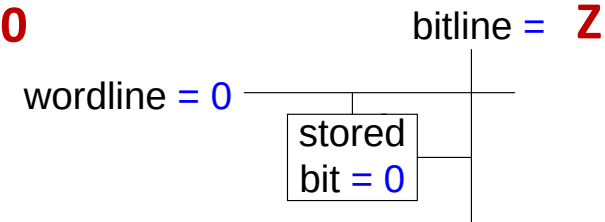
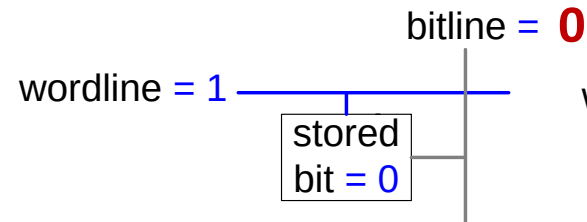
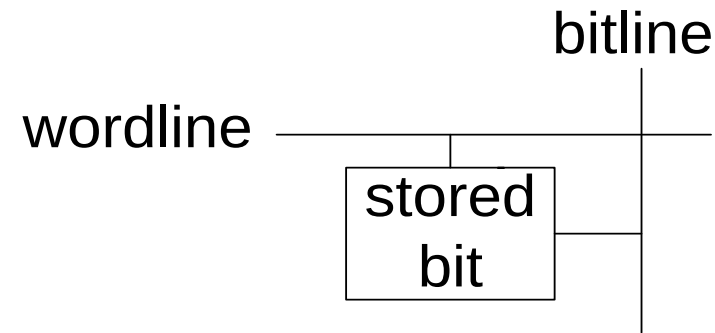


Address	Data			
11	0	1	0	depth ↑ ↓
10	1	0	0	
01	1	1	0	
00	0	1	1	
	width ←→			

Memory Arrays



Memory Array Bit Cells



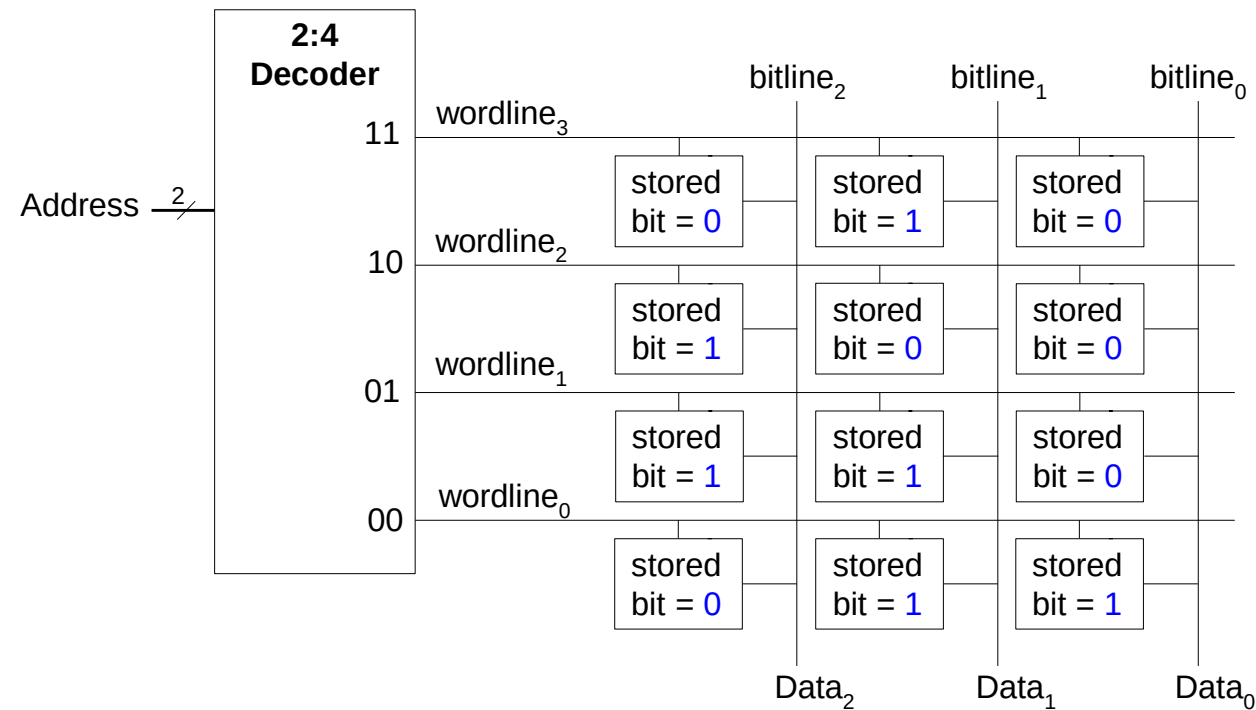
(a)

(b)

Memory Array

■ Wordline:

- Agisce come un enable
- Seleziona una riga nella memoria
- Corrisponde ad un unico indirizzo
- Solo una wordline per volta è attivata



Tipi di memoria

- Random access memory (RAM): **volatile**
- Read only memory (ROM): **nonvolatile**

RAM: Random

- **Volatile:** i dati sono persi quando il computer è spento
- Le operazioni di lettura e scrittura sono più veloci (rispetto alle ROM)
- E' la memoria primaria di un computer (DRAM)

E' chiamata *random access memory* perché il tempo di accesso ad una parola è lo stesso per ogni indirizzo (a differenza delle memorie ad accesso sequenziale come i nastri che si usavano ai tempi del C64)

ROM: Read Only Memory

- **Non volatile:** mantiene i dati anche quando il computer è spento
- La lettura è relativamente veloce ma la scrittura è lenta o semplicemente non è possibile scrivere
- Drives, flash memory, Bios etc. sono ROMs

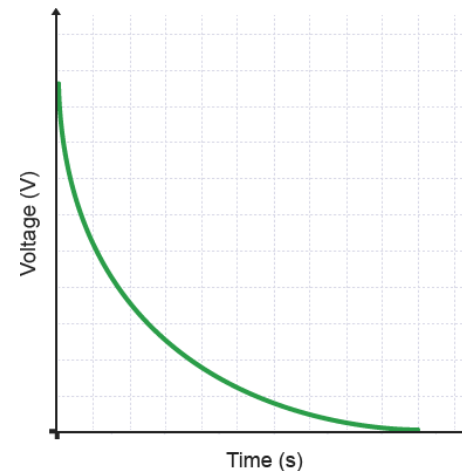
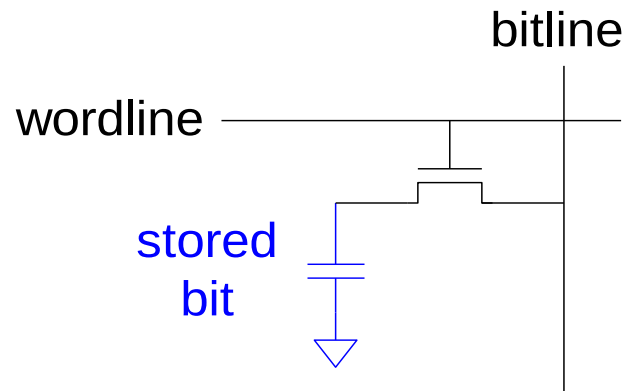
ROM sta per *read only* memory perché inizialmente questo tipo di memorie venivano “scritte” bruciando dei fusibili. Quindi, una volta configurate, non erano più manipolabili. Chiaramente questo non è più il caso per Flash memory e altri tipi di ROMs.

Tipi di RAM

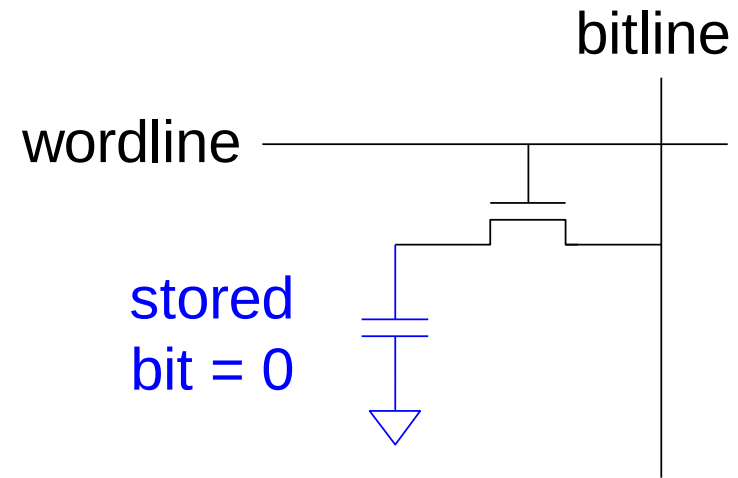
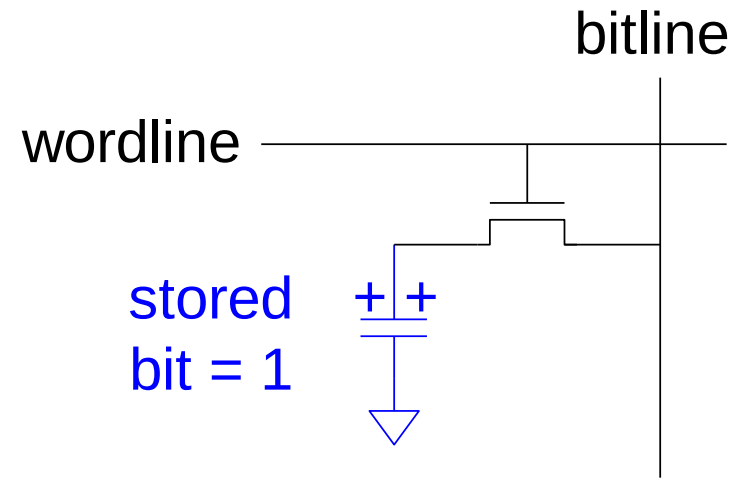
- **DRAM** (Dynamic random access memory)
- **SRAM** (Static random access memory)
- Si differenziano per le componenti usate per memorizzare i dati:
 - DRAM usa delle capacità
 - SRAM usa invertitori

DRAM

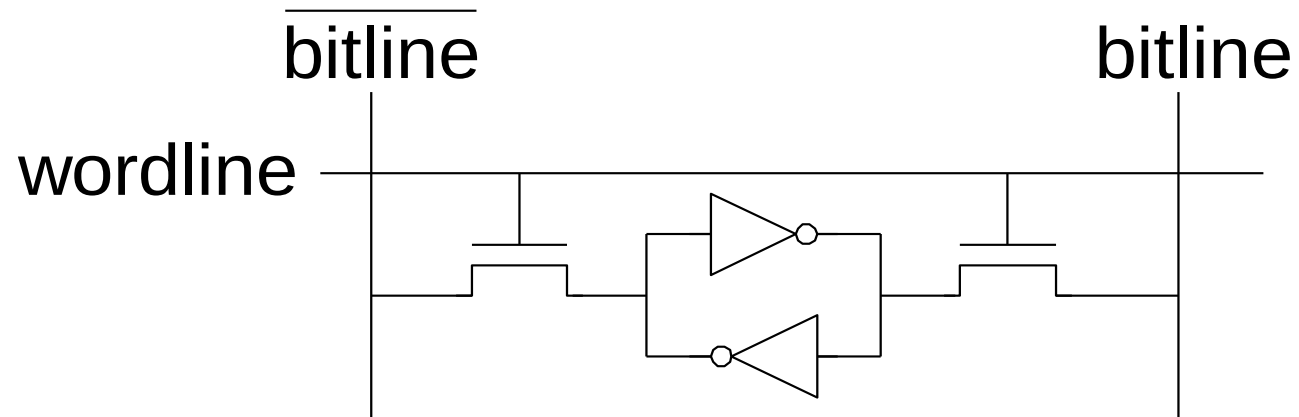
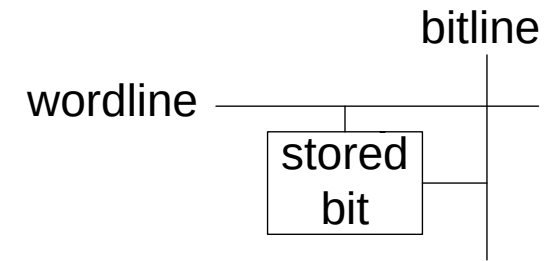
- i bit data sono immagazzinati in delle capacità
- *Dynamic* perché il valore di un bit deve essere “refreshed” (riscritto) periodicamente e anche dopo che è stato letto:
 - La perdita di carica di una capacità degrada il valore memorizzato
 - La lettura distrugge il valore letto



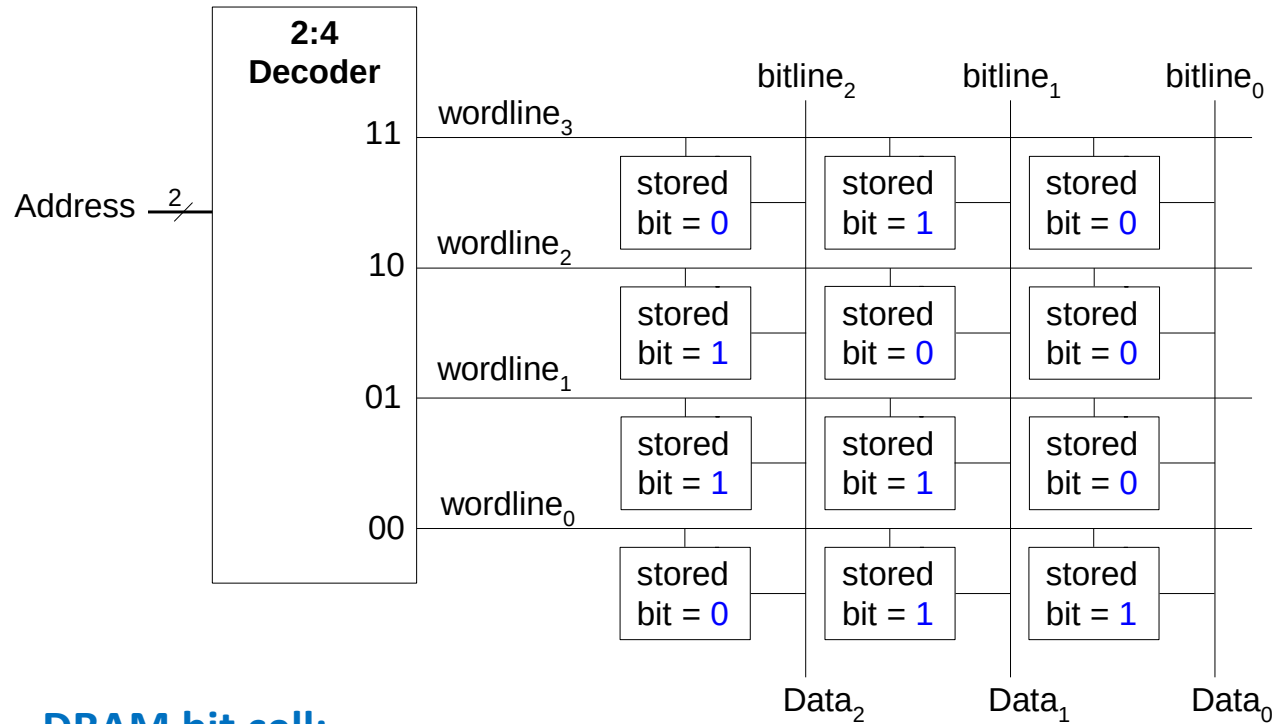
DRAM



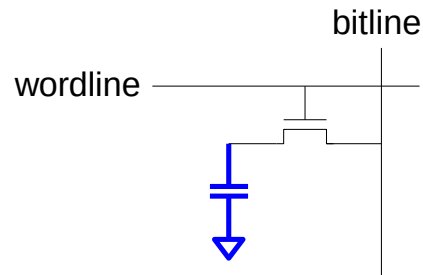
SRAM



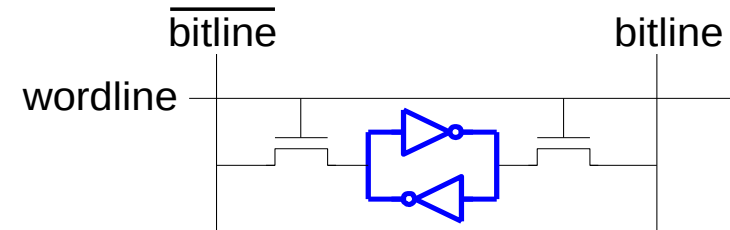
Memory Arrays Review



DRAM bit cell:



SRAM bit cell:



DRAM vs SRAM

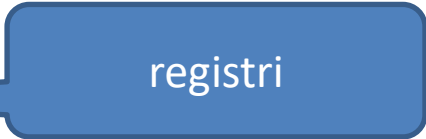
- DRAM è più lenta
- Scrittura e refresh (millisecondi) inducono un consumo maggiore di energia
- DRAM, più economiche

Memory Type	Transistors per Bit Cell	Latency
flip-flop	~20	fast
SRAM	6	medium
DRAM	1	slow

DRAM vs SRAM

- DRAM è più lenta
- Scrittura e refresh (millisecondi) inducono un consumo maggiore di energia
- DRAM, più economiche

Memory Type	Transistors per Bit Cell	Latency
flip-flop	~20	fast
SRAM	6	medium
DRAM	1	slow



registri

DRAM vs SRAM

- DRAM è più lenta
- Scrittura e refresh (millisecondi) inducono un consumo maggiore di energia
- DRAM, più economiche

Memory Type	Transistors per Bit Cell	Latency
flip-flop	~20	fast
SRAM	6	medium
DRAM	1	slow



cache

DRAM vs SRAM

- DRAM è più lenta
- Scrittura e refresh (millisecondi) inducono un consumo maggiore di energia
- DRAM, più economiche

Memory Type	Transistors per Bit Cell	Latency
flip-flop	~20	fast
SRAM	6	medium
DRAM	1	slow

memoria centrale

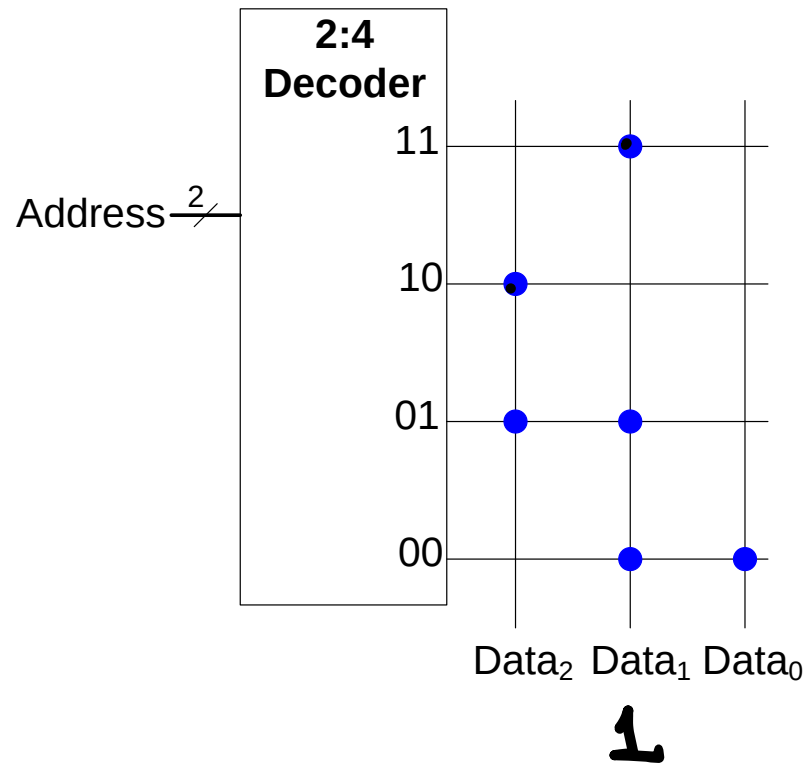
DDR SDRAM

- (DDR) Double data rate (S) synchronous (DRAM) dynamic random access memory
- Le operazioni di lettura e scrittura sono sincronizzate da un clock
- Le trasmissioni avvengono sia nel rising-edge del clock ($0 \rightarrow 1$) che nel che falling-edge ($1 \rightarrow 0$)
- In questo modo il data-bandwidth doppio rispetto alla frequenza di clock (double pumping)

DDR SDRAM

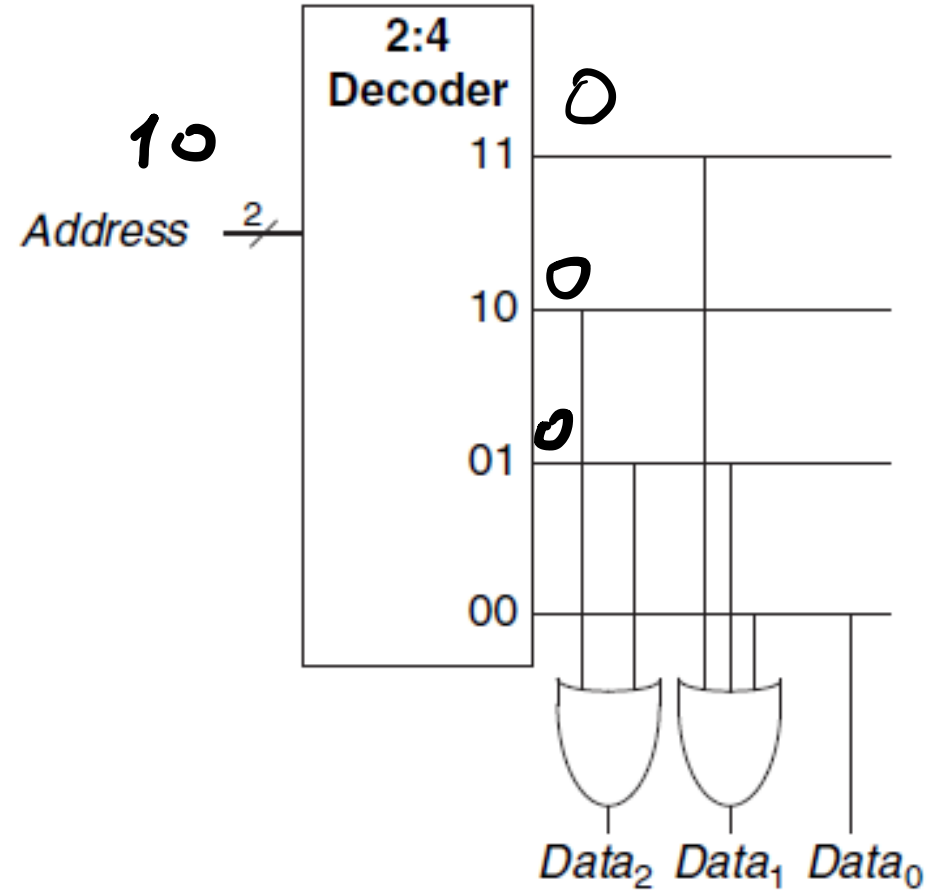
Standard name	Memory clock (MHz)	Cycle time ^{[4]} (ns)	Data rate (MT/s)	Peak transfer rate (MB/s)
DDR-200	100	10	200	1600
DDR-266	133.33	7.5	266.67	2133.33
DDR-333	166.67	6	333.33	2666.67
DDR-400	200	5	400	3200

ROM Storage

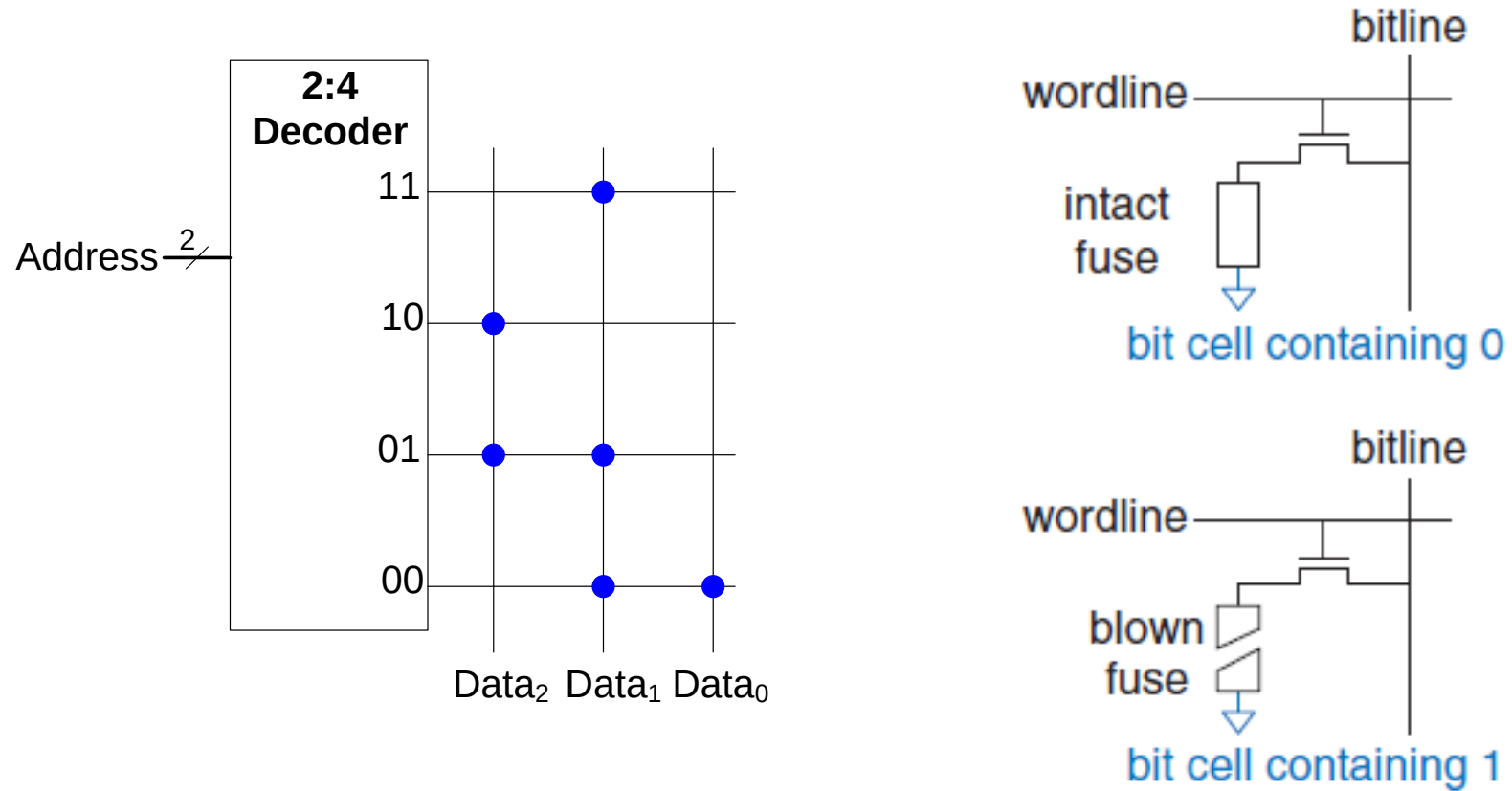


Address	Data			depth ↑ ↓
11	0	1	0	
10	1	0	0	
01	1	1	0	
00	0	1	1	
			width ←→	

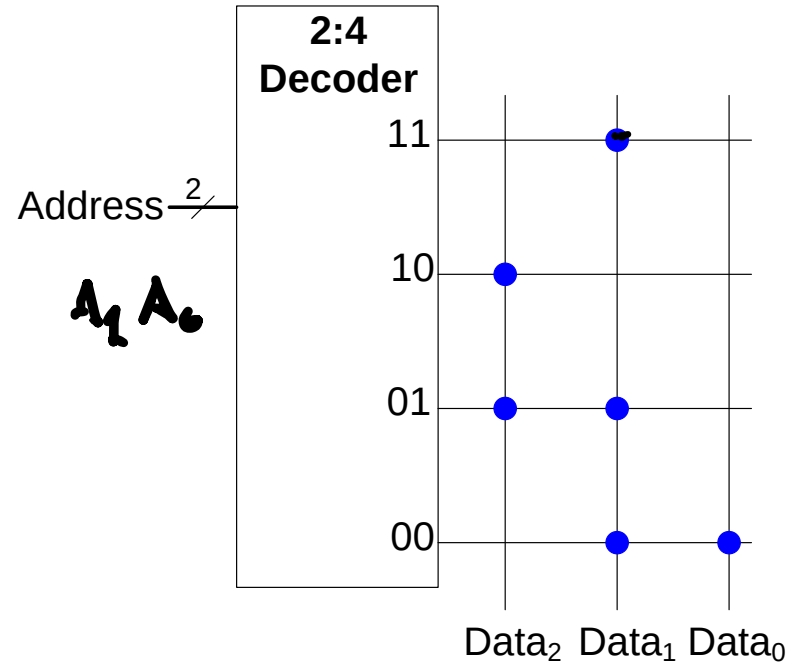
ROM via porte logiche



Fuse Programmable ROM



ROM Logic



$$Data_2 = A_1 \oplus A_0$$

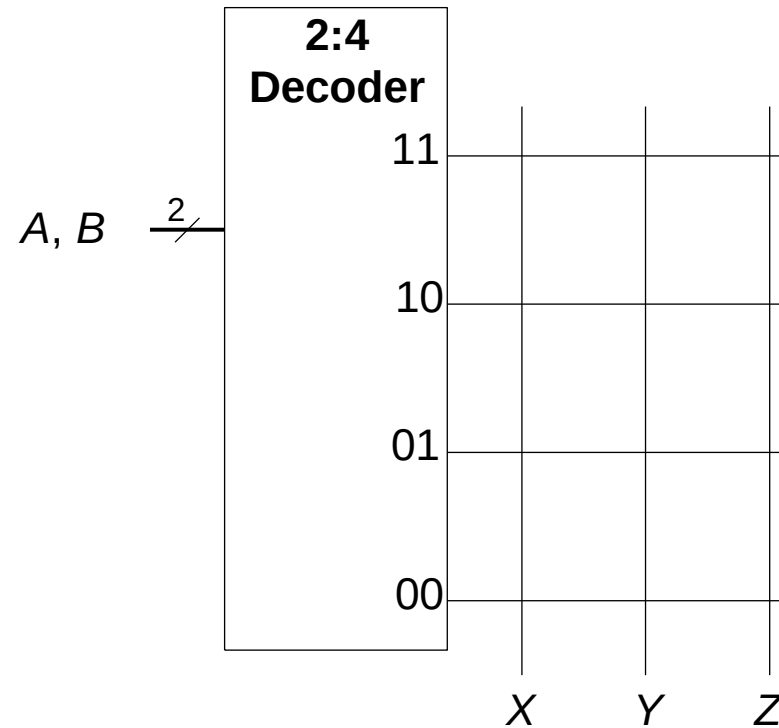
$$Data_1 = \overline{A_1} + A_0$$

$$Data_0 = \overline{A_1} \overline{A_0}$$

Esempio

Usare un $2^2 \times 3$ -bit ROM per implementare funzioni booleane:

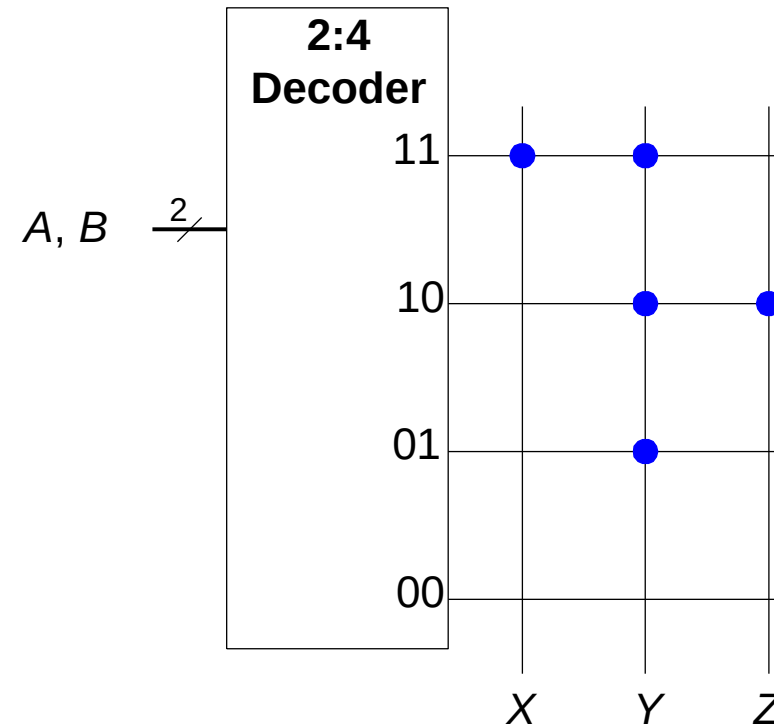
- $X = AB$
- $Y = A + B$
- $Z = A \overline{B}$



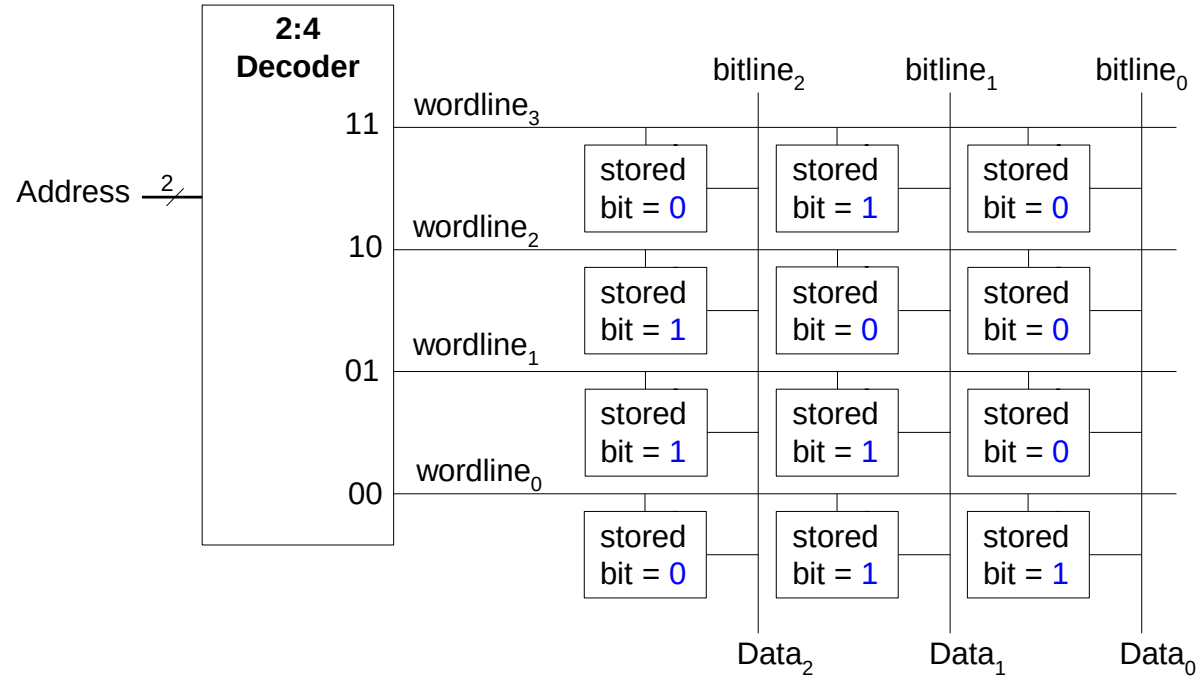
Esempio

Usare un $2^2 \times 3$ -bit ROM per implementare funzioni booleane:

- $X = AB$
- $Y = A + B$
- $Z = A \overline{B}$



Lookup tables (LUTs)



$$Data_2 = A_1 \oplus A_0$$

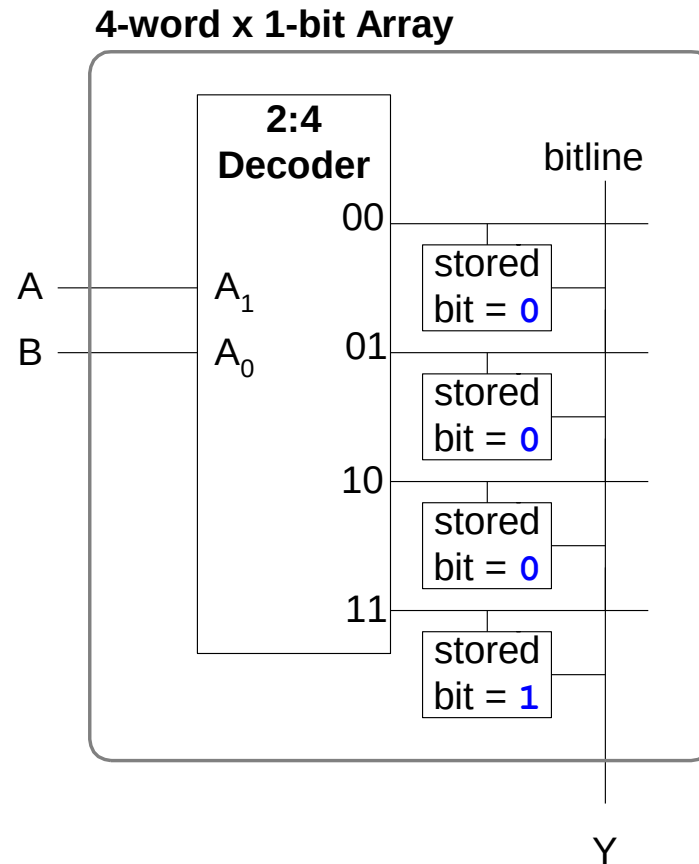
$$Data_1 = \overline{A_1} + A_0$$

$$Data_0 = \overline{A_1} \overline{A_0}$$

Logiche con memory array

Sono chiamate *lookup tables* (LUTs): si “guardano” gli output per ogni combinazione di input (indirizzo)

Truth Table		
A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1



Logic Arrays

- **PLAs** (Programmable logic arrays)
 - AND array seguito da un OR array
 - solo circuiti combinatori (SOP)
 - Le connessioni interne sono fisse
- **FPGAs** (Field programmable gate arrays)
 - Array elementi logici (LE)
 - Circuiti combinatori e sequenziali
 - Programmable internal connections

PLAs

- $X = \overline{A}BC + ABC\overline{C}$
- $Y = A\overline{B}$

